

Neighborhood and Community Automation System (CommuneX)

Project Team

Sheraz Tariq	21i-1188
Umair Ahmed	21i-1182
Aneeb ur Rehman	20i-1886

Session 2021-2025

Supervised by

Dr. Isma ul Hassan



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	Property buying and selling	2
1.3.2	Event Management	2
1.3.3	Decentralized voting system	2
1.3.4	Shared Resources	3
1.3.5	User Authentication and Role Management	3
1.3.6	Communication and Announcement Module	3
1.3.7	Admin Dashboard	3
2	Project Requirements	5
2.1	Use-case	5
2.1.1	Property buying and Selling	5
2.1.2	Decentralized Voting	8
2.1.3	Resource Sharing	11
2.1.4	Event Management	14
2.1.5	Communication and Announcements	17
2.1.6	Maintenance	19
2.1.7	User Authentication and Role Management	22
2.1.8	Admin Dashboard	24
2.1.9	Use Case Diagram	27
2.2	Functional Requirements	28
2.2.1	Property Buying and Selling	28
2.2.2	Decentralized voting	29
2.2.3	Shared Resources	29
2.2.4	Event Management	30
2.2.5	Community News and Announcements	31
2.2.6	Maintenance	31
2.2.7	User Authentication and Role Management	32
2.2.8	Admin Dashboard	33

2.3	Non-Functional Requirements	34
2.3.1	Reliability	34
2.3.2	Usability	34
2.3.3	Performance	34
2.3.4	Security	35
3	System Overview	37
3.1	Architectural Design	37
3.1.1	System Architecture Design	37
3.2	Design Models	39
3.3	Data Design	66
3.3.1	User Management	66
3.3.2	Property Management	66
3.3.3	Decentralized Voting system	66
3.3.4	Resource Sharing	66
	References	69

List of Figures

2.1	Use Case Diagram for CommuneX	27
3.1	System Architecture diagram	38
3.2	Property Listing	39
3.3	Purchase Property	39
3.4	Resource Booking	40
3.5	Resource Listing	40
3.6	View and Search Property	41
3.7	view and search resource	41
3.8	voting creation	42
3.9	Voting Results	42
3.10	voting	43
3.11	Event Management	43
3.12	Event Recommendation	43
3.13	Community News and Announcements	44
3.14	Private Messaging	44
3.15	Submit Maintenance	44
3.16	Track Progress	45
3.17	User Registration	45
3.18	User Login and Role Assignment	46
3.19	Admin Dashboard	46
3.20	Manage Users and roles	47
3.21	Data Flow Diagram for CommuneX	47
3.22	Data Flow Diagram for CommuneX	48
3.23	Data Flow Diagram for CommuneX	48
3.24	Property Listing Activity diagram	49
3.25	Property View and Search Activity diagram	50
3.26	Purchase Property Activity diagram	51
3.27	Voting Creation Activity diagram	52
3.28	Voting Activity diagram	53
3.29	Voting Results Activity diagram	54
3.30	Resource List Activity diagram	55
3.31	Resource Search and view Activity diagram	56

3.32 Resource Booking Activity diagram	57
3.33 Communication and Announcements Activity diagram	58
3.34 Maintenance Activity diagram	59
3.35 Resource List Activity diagram	60
3.36 Admin activity diagram	61
3.37 User Management activity diagram	62
3.38 Register User activity diagram	63
3.39 User role and assignment activity diagram	64
3.40 State Transition diagram	65

List of Tables

2.1	Property Buying and selling	5
2.2	Property Searching and Viewing	6
2.3	Property Purchase	7
2.4	Voting Proposal Creation	8
2.5	Voting	9
2.6	Voting Results	10
2.7	Resource Listing	11
2.8	Resource Searching and Viewing	12
2.9	Resource Booking	13
2.10	Event Management	14
2.11	Event Recommendation	16
2.12	Community News and Announcements	17
2.13	Private Messaging	18
2.14	Submit Maintenance	19
2.15	Track Progress	21
2.16	User Registration	22
2.17	User Login and Role Assignment	23
2.18	System Overview and Monitoring	24
2.19	Manage Users and Roles	25

Chapter 1

Introduction

This chapter will give the overview of the project. We will be automating the community and neighborhood system which gives the comprehensive solutions to streamline the community operations. Besides, there is a lot of paper work involved in communities, most of the communities had difficulties to remain engaged in the society, seamless transaction system and the lack of events arrangements which would cause problems for the communities. The project solution involves multiple functionalities to achieve the project objective.

1.1 Problem Statement

The Community and Neighbor automation system handles the challenges of the residential communities. Enhanced communication between residents and management of the society/community and efficient operations are critical as neighborhoods increases in size. For tasks like property transactions, maintenance requests, event management, and communication, current systems frequently rely on manual processes. This results in inefficiencies, a lack of transparency, and delays.

Our solution aims to combine different community tasks into a single platform, enabling smoother operations. Residents will be able to book shared resources, report issues, purchase and sell real estate securely, and take part in community debates in addition to submitting maintenance requests. In order to ensure transparency in financial transactions and decision-making, it also integrates blockchain technology for safe transactions and voting procedures. Our solution eliminates communication barriers, reduces operational delays, and guarantees an open, safe, and connected neighborhood environment by automating these fundamental tasks and centralizing community activities.

1.2 Scope

The Scope of Neighborhood and Community automation system involves the comprehensive platform that integrate with various modules to improve the community operations. The functionalities involve the user authentication, seamless communication platform, incident reporting and community voting system through block chain. Moreover, the property documents will be handled and created using a blockchain. The blockchain system will provide the safe transaction system for the communities and the real time notification. Event management include managing the events in the community as well as the resource would be shared in the communities to enhance the automated system. The community will do their payments and transaction using blockchain. The project aims to enhance communication, efficiency and automate the community on advance culture or in a global village.

1.3 Modules

1.3.1 Property buying and selling

Description: This module will simplify the processes and procedures of the properties whether it is the buying or selling, money transactions or just viewing the prices of the plots and houses.

1. **Feature 1:** Buying and selling
2. **Feature 2:** Block chain based documentations

1.3.2 Event Management

Description: This module will help the community members to book any sort of events, places for birthday parties or just a personalized event. Our model will also recommend the events to the members.

1. **Feature 1:** Event Booking
2. **Feature 2:** Event recommendation.

1.3.3 Decentralized voting system

Our platform will have the functionality of a voting system which will let the users to record their say in the important decisions of the community. The voting process will be smoother and efficient and will be kept secure.

1. **Feature 1:** Block chain based voting system

1.3.4 Shared Resources

Description: This module will enable the residents to share each other resources. The members of the society will be able to request different services from their fellow neighbors. This will increase the community interaction and promotes resources efficiency.

1. **Feature 1:** Offer or Request services from neighbors.

1.3.5 User Authentication and Role Management

Description: This module will ensure a smooth and secure access to our system. Different functionalities such as sign up/login, password recovery, permissions will be managed.

1. **Feature 1:** User Registration and login
2. **Feature 2:** Credentials based on Roles

1.3.6 Communication and Announcement Module

Description: This module will facilitate a well-organized connection between the community members. Essential information will be conveyed to others and a proper discussion could be made on announcements.

1. **Feature 1:** Community news and announcement
2. **Feature 2:** Private Messaging

1.3.7 Admin Dashboard

Description: The admin dashboard will have access to the advanced tools that will help manage daily community operations.

1. **Feature 1:** Comprehensive Overview
2. **Feature 2:** User Management

Chapter 2

Project Requirements

2.1 Use-case

2.1.1 Property buying and Selling

Table 2.1: Property Buying and selling

Section	Content
Use Case Name	List Property
Scope	Community Automation System
Level	High-level
Primary Actor	Resident
Stake Holders and Interests	Resident who wants to buy or sell the property, Admin for approval, and buyers to access the listing.
Pre-condition	The resident must login to the automation system.
Post-Condition	Property listed successfully and property listed for purchase.
Main Success Scenario	
User Action	System Response

The resident navigates to the property managing section.	After clicking on the "Property Management", the system will display a management screen.
The resident selects the "List Property" option from the list appearing on the screen.	The community automation system asks the property details to enter into the system.
The resident enters the required details.	
The resident confirms by clicking on the "Confirm" button.	
Admin checks the details, verifies the authenticity of the property, and approves the listing.	The system successfully registers the property for sale and displays a confirmation screen.
Alternate Scenario	In case the admin selects "NO" option, the system will redirect the resident back to the homepage.
Exception Scenario	The system will reject the listing if any details are invalid.

Table 2.2: Property Searching and Viewing

Section	Content
Use Case Name	Search and View Property
Scope	Allow the residents to search and view property.
Level	High-goal
Primary Actor	Resident
Stake Holders and Interests	Resident, Sellers, and the admin for oversight.
Pre-condition	Resident must be logged in into the community automation System.

Post-Condition	The resident has detailed information about the available properties.
Main Success Scenario	
User Action	System Response
User will click on the “Search” button to search the property.	The system will display a search bar.
The resident enters the details of the property to search and clicks search.	The system will respond according to the search details and suggest according to the details.
The resident will view the property according to his/her search details.	A crystal-clear view of the property with the details displayed on the screen.
The resident clicks to view further details.	The system asks the user to enter the PIN for verification purposes.
Alternate Scenario	The resident can search the results on additional criteria.
Exception Scenario	The system will display an error message if there is no result matching the search criteria.

Table 2.3: Property Purchase

Section	Content
Use Case Name	Purchase Property
Scope	Allow residents to purchase the listed properties.
Level	High-level
Primary Actor	Resident
Stakeholders and Interests	Resident who buys, Seller who receives payment, and the admin for the payment approval.

Pre-condition	The resident must be logged in to the automation system.
Post-condition	The property is sold and the transaction recorded securely.
Main Success Scenario	
User Action	Community Automation System
The Resident clicks on the “Purchase property” button.	The community and neighborhood automation system will display a list of properties for sale.
The resident clicks on the property according to his/her budget or requirements.	The system will show the details of the property.
The resident goes for the buy and clicks on the buy.	The system will take the resident to the transaction form details.
The resident enters the details and clicks the checkout.	The system will check the transaction and send the confirmation to the admin.
The admin checks the transaction details and accepts the purchase based on transactions.	The system will send a notification to the resident on successful purchase of the property.
Alternate Scenario	The resident can cancel the transaction before final confirmation.
Exception Scenario	The system will reject the transaction if the payment fails or the property is no longer available.

2.1.2 Decentralized Voting

Table 2.4: Voting Proposal Creation

Section	Content
Use Case Name	Create Voting poll

Scope	Allow admin to create voting polls for community decisions.
Level	High-level
Primary Actor	Admin
Stakeholders and Interests	Admin voting polls creator, Residents for vote, Community for decision making.
Pre-condition	The admin must have been logged into the system.
Post-condition	The voting is published for and available to all residents.
Main Success Scenario	
Admin	System Response
The admin navigates to the voting section.	The system will display the voting section.
The admin clicks on the create new poll.	The system will show the form of the new poll.
The admin fills the voting poll details and submits the proposal.	The system will create a poll.
	A message appears on the admin screen “voting creates successfully”.
Alternate Scenario	The admin can save voting as draft and complete it later.
Exception Scenario	The system rejects the proposal if the details are invalid.

Table 2.5: Voting

Section	Content
Use Case Name	Cast Vote
Scope	Allow the residents to vote on community proposals.

Level	High-level
Primary Actor	Resident
Stakeholders and Interests	Resident is voter and admin for oversight.
Pre-condition	The resident must login into the system.
Post-condition	The voter vote is counted and results are updated.
Main Success Scenario	
Resident	System Response
The resident navigates to the voting section.	The system will display the voting section and display active voting section.
The resident clicks on the active voting.	The system displays the details of the voting and offers to vote.
The resident votes for his own decision and submits the vote.	The system will display the vote review details and ask for confirmation.
The resident confirms the vote.	The system displays your vote is successfully noted.
Alternate Scenario	The voter cannot change his request once it's registered.
Exception Scenario	The system rejects the vote if the voter has already voted or the voting period has ended.

Table 2.6: Voting Results

Section	Content
Use Case Name	voting results
Scope	Allow the admin and residents to view the results.
Level	High-level

Primary Actor	Resident, Admin
Stakeholders and Interests	Resident is voter and admin for oversight.
Pre-condition	The voting period must be ended.
Post-condition	The results are ready for view.
Main Success Scenario	
User Action	System Response
The user navigates to the voting section.	The system will display the voting section and display active voting section.
The user selects the voted proposal.	The system displays the details of the proposal voted.
The user can view the results.	The system will display the vote results.
Alternate Scenario	The admin can export the results for further analysis.
Exception Scenario	The system displays an error message if the results are not available.

2.1.3 Resource Sharing

Table 2.7: Resource Listing

Section	Content
Use Case Name	List Resource
Scope	Residents list the resources for community use.
Level	High-level
Primary Actor	Resident

Stakeholders and Interests	Resident owner of resource, admin for approval and community users.
Pre-condition	The resident must login into the system.
Post-condition	The resource is listed and available for viewing and booking by other residents.
Main Success Scenario	
User Action	System Response
The resident navigates to the shared resources section.	The system will display the resource section.
The resident clicks on the add new resource button.	The system displays the detail of the resource form to fill.
The resident enters the details and clicks to add the resource.	The resident submits the resource.
	The system will check the status of the resource added and send to the admin for approval.
The admin confirms the resource according to the requirements.	The system displays a message that your resource is added successfully.
Alternate Scenario	The resident can save the resource or complete whenever he wants.
Exception Scenario	The system rejects the resource if the details are incomplete or invalid.

Table 2.8: Resource Searching and Viewing

Section	Content
Use Case Name	Search and View Resource
Scope	Allow the residents to search and view the resources.

Level	High-level
Primary Actor	Resident
Stakeholders and Interests	Resident and admin for oversight.
Pre-condition	The resident must login into the system.
Post-condition	The resident has access to detailed information about available resources.
Main Success Scenario	
Resident	System Response
The resident navigates to the search bar.	The system will display the search bar and allow the user to enter search prompt.
The resident enters the details of the resources he needed.	The system will display the results according to the search results.
The resident selects the resource he needs.	
The resident clicks on the resource.	The system will display the details of the resource.
Alternate Scenario	The residents can filter the search results based on additional criteria.
Exception Scenario	The system shows an error if the resources don't match with the search results.

Table 2.9: Resource Booking

Section	Content
Use Case Name	Book Resource
Scope	Allow the residents to book shared resources for use.

Level	High-level
Primary Actor	Resident
Stakeholders and Interests	Resident is user, Owner of the resource and admin for approval.
Pre-condition	The resident must log in to the system.
Post-condition	The resident will be able to book the resource according to usage.
Main Success Scenario	
User Action	System Response
The resident navigates to the resource section.	The system will display the resource section.
The resident clicks on the resource to book.	The system displays resource availability and asks the user to confirm.
The resident confirms the booking.	
The resident submits the booking.	The admin approves the resource.
	The system sends confirmation to the admin.
	The system displays that the resource is booked successfully.
Alternate Scenario	The resident can cancel the booking before confirmation.
Exception Scenario	The system rejects the booking if the resource is not available or the booking is invalid.

2.1.4 Event Management

Table 2.10: Event Management

Section	Content
----------------	----------------

Use Case Name	Manage Event
Scope	Community Automation System
Level	User Goal
Primary Actor	Community Member
Stakeholders	Community member, Event Management Team, and System administrator
Preconditions Venue and resources must be available in the system for booking.	Members must have logged in.
Post Conditions - Venue marked as reserved for that specific time slot.	- Event Successfully Booked and confirmation sent to the user
Main Success Scenario	
User Action	System Response
Member logs in to the system	System validates credentials.
Member navigates to the booking page	System displays available venues, dates, and event options.
Member selects a venue, event type, and date	System checks availability and confirms if the slot is available.
Alternate Scenarios	If the member chooses a date that is partially available, the system suggests alternate available time slots or nearby venues.

Exception Scenarios	If no venues are available for the selected date, the system prompts the member to choose another date or provides a list of other available venues.
----------------------------	--

Table 2.11: Event Recommendation

Section	Content
Use Case Name	Recommend Event
Scope	Community Automation System
Level	User Goal
Primary Actor	Community Member
Stakeholders	Community member, Event Management Team, and System administrator
Preconditions	Member must have selected the preferred event recommendations, e.g., any sort of sports events.
Post Conditions Members can view the details.	The members are presented with relevant events.
Main Success Scenario	
User Action	System Response
Member logs in to the system	System validates credentials.
Members navigate to the recommended events.	
Members view a list of events.	System displays event recommendations based on preferences.

Member selects a recommended event	The system displays event details, including time, venue, and cost.
Alternate Scenarios	If the member has no previous interactions, the system recommends promotional events currently advertised by the event organizers. These promotional events are displayed based on event priority rather than member preferences.
Exception Scenarios	If the recommended event is fully booked or canceled, the system notifies the member and suggests similar events.

2.1.5 Communication and Announcements

Table 2.12: Community News and Announcements

Section	Content
Use Case Name	News and Announcements
Scope	Community Automation System
Level	User Goal
Primary Actor	Community Member
Stakeholders	Community member, Community Admin, and System administrator
Preconditions - Announcements or news should be created by moderators/admins and exist in the system database.	- The member must be logged in.
Post Conditions	- The member receives the announcement or news.
Main Success Scenario	

User Action	System Response
Member logs in to the system	System validates credentials.
Members navigate to the Announcements Section.	System displays the full content of the announcement, including details like time, date, and any attachments.
Members read the announcement.	
Alternate Scenarios	If discussions are allowed on announcements, the system enables a comments section where the member can post feedback or discuss the announcement.
Exception Scenarios	The previous announcement will be kept showing unless a new one is posted.

Table 2.13: Private Messaging

Section	Content
Use Case Name	Private Messaging
Scope	Community Automation System
Level	User Goal
Primary Actor	Community Member
Stakeholders	Community member, Community Admin, and System administrator
Preconditions	- Member must be logged in.
- The system should provide access to a list of other members who can be messaged.	

Post Conditions	Messages are successfully sent, received, and stored.
Main Success Scenario	
User Action	System Response
Member logs in to the system	System validates credentials.
Members navigate to Private Messaging.	The system displays a list of recent conversations and available members to message.
Member selects a recipient and starts typing a message.	System displays a text box and enables typing.
Member sends the message.	System sends the message to the recipient and confirms successful delivery.
Alternate Scenarios	If the recipient is offline, the system queues the message.
Exception Scenarios	If the system does not find the intended recipient (e.g., the user has deactivated their account), the system alerts the sender and does not allow sending the message.

2.1.6 Maintenance

Table 2.14: Submit Maintenance

Section	Content
Use Case Name	Submit Maintenance
Scope	Community Automation System
Level	User Goal
Primary Actor	Community Member

Stakeholders	Community member, community admin, and System administrator
Preconditions - Maintenance staff must be available to receive and address the queries.	- The member must be logged into the system.
Post Conditions - The system assigns a tracking number to the issue, and the maintenance team receives a notification.	- The query or incident report is successfully submitted.
Main Success Scenario	
User Action	System Response
Member logs in to the system	System validates credentials.
Members navigate to submit query.	System displays a form to submit a maintenance query or incident report.
Member fills in the details of the issue (e.g., broken streetlight, infrastructure damage).	System accepts the input and validates the data.
Member submits the query.	System assigns a tracking number to the query and confirms successful submission.
Alternate Scenarios	If the member attaches images or files (e.g., pictures of the issue), the system validates the attachments and includes them with the query.
Exception Scenarios	If the query fails to be submitted due to a network error or system malfunction, the system notifies the member and allows a retry.

Table 2.15: Track Progress

Section	Content
Use Case Name	Track Progress
Scope	Community Automation System
Level	User Goal
Primary Actor	Community Member
Stakeholders	Community member, community admin, and System administrator
Preconditions	- The member must have submitted a query or incident report.
Post Conditions - The system logs each progress update for future reference.	- The member is able to view the current status of their submitted query.
Main Success Scenario	
User Action	System Response
Member logs in to the system	System validates credentials.
Members navigate to Track Progress.	System displays a list of all submitted queries and their current status.
Member selects a specific query to view details.	System displays detailed progress updates (e.g., "in progress," "awaiting materials," "completed").

Alternate Scenarios	• If the query is taking longer than expected, the system provides an estimated completion time and reason for delay (e.g., awaiting approval, parts needed). • If the issue has been resolved, the system allows the member to provide feedback on the service or reopen the query if the issue persists.
Exception Scenarios	If the member selects a query that has been closed or canceled, the system shows the final status and reasons for closure without allowing further updates.

2.1.7 User Authentication and Role Management

Table 2.16: User Registration

Section	Content
Use Case Name	User Registration
Scope	Community Automation System
Level	High-level
Primary Actor	Resident
Stakeholders and Interests	Resident: Wants to create an account and gain access to the system.
Pre-condition	The resident is not registered in the system.
Post-condition	The resident is successfully registered and can log in.
Main Success Scenario	
User Action	System Response

The resident navigates to the registration page.	
The resident fills out personal details (name, email, address, etc.).	The system validates the entered details.
	If the email is not taken already, the account of the resident will be created and the resident is registered.
Alternate Scenario	The resident can update their registration details before registering.
Exception Scenario	If the email is already taken, the system will notify the user that the email is already taken.

Table 2.17: User Login and Role Assignment

Section	Content
Use Case Name	User Login and Role Assignment
Scope	Community Automation System
Level	High-goal
Primary Actor	Resident/admin
Stakeholders and Interests	• Resident: Wants to log in and access the system based on assigned role. • Admin: Enters admin credentials to access the admin interface.
Pre-condition	The resident must have an account in the automation system.
Post-condition	The resident is successfully authenticated and granted access based on their role (Resident, Admin).
Main Success Scenario	

User Action	System Response
The resident navigates to the login page.	
The resident enters valid credentials (username and password).	The system validates the credentials.
The resident/admin accesses their dashboards.	The system redirects the resident/admin to the dashboard based on their role.
Alternate Scenario	If the residents forget the password of their accounts, they can use the "forget password" option.
Exception Scenario	The system will prompt the resident to try again and provide an error notice if the credentials are invalid.

2.1.8 Admin Dashboard

Table 2.18: System Overview and Monitoring

Section	Content
Use Case Name	System Overview and Monitoring
Scope	Allow residents to purchase the listed properties.
Level	High-level
Primary Actor	Admin
Stakeholders and Interests	• Admin: Needs to monitor system activities, users, and system performance.
Pre-condition	Admin is logged in and has access to the dashboard.

Post-condition	Admin successfully views a detailed overview of community operations, including real-time data on property listings, maintenance requests, event management, and user activity.
Main Success Scenario	
User Action	Community Automation System
The Resident clicks on the “Purchase property” button.	The system displays a dashboard with various widgets showing the status of different community operations (e.g., property sales, shared resources usage, active maintenance requests, upcoming events, etc.).
Admin clicks on the "Overview" widget for a high-level summary of all active and pending tasks.	
Admin can drill down into specific areas (e.g., pending maintenance requests) by clicking on detailed views.	The system provides visual representations such as graphs, charts, and lists for easy interpretation of data.
Admin logs out or navigates to another module after reviewing the data.	
Alternate Scenario	Admin can customize the dashboard view by adding or removing widgets based on their preferences.
Exception Scenario	The system will indicate the error and advise trying again later if there is a problem that prevents data from being retrieved.

Table 2.19: Manage Users and Roles

Section	Content
Use Case Name	Manage Users and Roles

Scope	Community Automation System
Level	High-level
Primary Actor	Admin
Stakeholders and Interests	Admin: Needs to manage user accounts, assign roles, deactivate users, and ensure proper role-based access control.
Pre-condition	Admin must have access to the "User Management" section of the dashboard.
Post-condition	User roles are successfully updated, and inactive or non-compliant users are deactivated.
Main Success Scenario	
Admin	System Response
Admin logs into the system and navigates to the "User Management" section on the dashboard.	
Admin filters users by role, activity status, or registration date.	The system displays a list of all registered users along with their current roles (Resident, Admin).
Admin selects a user and views detailed profile information.	
Admin clicks "Edit" to update the user's role (e.g., promoting a Resident to Moderator) or deactivate the account.	
Alternate Scenario	Admin can sort users by various criteria (e.g., role, last login date) to quickly find specific accounts.

Exception Scenario

If the system encounters an error while updating user roles, it displays an error message and prompts the admin to try again.

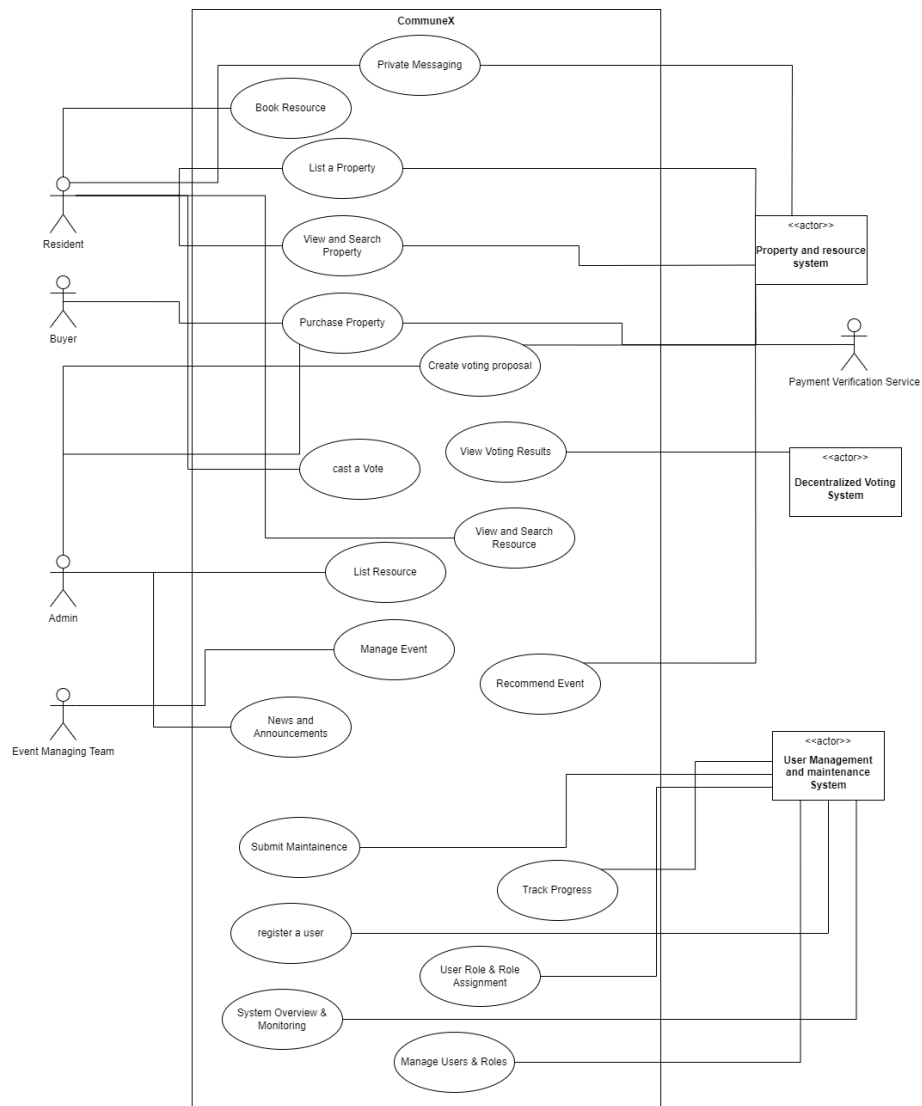
2.1.9 Use Case Diagram

Figure 2.1: Use Case Diagram for CommuneX

2.2 Functional Requirements

2.2.1 Property Buying and Selling

Following are the requirements for Property Buying and Selling:

1. **FR1** The system shall allow resident to list their properties for sale.
2. **FR2** The system shall validate all the fields in the listing form.
3. **FR3** The system shall stores the property details in database.
4. **FR4** The system shall send notifications to the admin for the approval.
5. **FR5** The system shall upload the property details after admin approval.
6. **FR6** The system shall allow the residents to search for the property based on price, location and size.
7. **FR7** The system shall display the resident search results.
8. **FR8** The system shall allow the residents to view the details of the property(photos and Descriptions).
9. **FR9** The system shall allow residents to save properties.
10. **FR10** The system shall provide filters to improve the search results.
11. **FR11** The system shall provide filters to improve the search results.
12. **FR12** The system shall display the property details and review for sale.
13. **FR13** The system shall guide the residents for the payment processes
14. **FR14** The system shall record the transaction on the block chain for the confidentiality and security
15. **FR15** The system shall guide the residents for the payment processes
16. **FR16** The system shall use the blockchain for the secure payments
17. **FR17** The system shall store the property papers on the block chain.

2.2.2 Decentralized voting

Following are the requirements for Decentralized voting:

1. **FR1** The system shall allow the admin to create the voting polls for community decisions.
2. **FR2** The system shall validate all the fields in the voting form.
3. **FR3** The system shall store the data in the database.
4. **FR4** The system shall notify all the residents on creating a new voting poll.
5. **FR5** The system shall validate all the fields in the voting form.
6. **FR6** The system shall deliver the voting proposal to the residents.
7. **FR7** The system shall allow the residents to view the active voting proposals.
8. **FR8** The system shall allow residents to cast their vote.
9. **FR9** The system shall record the voting on blockchain for security and transparency.
10. **FR10** The system shall not allow to vote again on the same proposal.
11. **FR11** The system shall allow the admin and residents to view their vote.
12. **FR12** The system shall calculate the accurate results of voting.
13. **FR13** The system shall allow admin to announce the voting results.
14. **FR14** The system shall conclude the voting results and make available to the residents.

2.2.3 Shared Resources

Following are the requirements for Shared Resources:

1. **FR1** The system shall allow residents to list their resources for the community use.
2. **FR2** The system shall validate the fields in the resources form.
3. **FR3** The system shall store the details in the database.
4. **FR4** The system shall notify the admin to approve resource listing.
5. **FR5** The system shall list the resources on admin approval.
6. **FR6** The system shall allow residents to search for resources based on the required criteria.

7. **FR7** The system shall display the search results.
8. **FR8** The system shall allow the residents to view the details of each resource they required.
9. **FR9** The system shall allow resident to save the resources.
10. **FR10** The system shall provide filters for easy search.
11. **FR11** The system shall allow residents to book a resource.
12. **FR12** The system shall display resource details and allow residents to review the resource.
13. **FR13** The system shall allow residents to decide their booking period.
14. **FR14** The system shall notify admin to review and approve the resource.

2.2.4 Event Management

Following are the requirements for Event Management:

1. **FR1** The system shall validate user credentials during login.
2. **FR2** The system shall display available venues, event dates, and options for booking.
3. **FR3** The system shall check and confirm the availability of selected venues.
4. **FR4** The system shall send a booking confirmation to the user upon successful reservation.
5. **FR5** The system shall suggest alternate time slots or nearby venues if the selected option is partially available.
6. **FR6** The system shall notify the user if no venues are available for the selected date and suggest other options.
7. **FR7** The system shall validate user credentials during login.
8. **FR8** The system shall store user event preferences for future recommendations.
9. **FR9** The system shall display event recommendations based on the user's preferences.
10. **FR10** The system shall suggest promotional events if no personalized recommendations are available.
11. **FR11** The system shall notify users if a recommended event is fully booked or canceled and provide similar event alternatives.

2.2.5 Community News and Announcements

Following are the requirements for Community News and Announcements:

1. **FR1** The system shall validate user credentials during login.
2. **FR2** The system shall display announcements, including the full content with time, date, and attachments.
3. **FR3** The system shall optionally enable a comments section for users to provide feedback on announcements.
4. **FR4** The system shall continue displaying the previous announcement until a new one is posted.
5. **FR5** The system shall validate user credentials during login.
6. **FR6** The system shall display a list of available members to message.
7. **FR7** The system shall allow users to send, receive, and store messages, confirming successful delivery.
8. **FR8** The system shall queue messages if the recipient is offline.
9. **FR9** The system shall notify the sender if the intended recipient is not available or has deactivated their account.

2.2.6 Maintenance

Following are the requirements for Maintenance:

1. **FR1** The system shall validate user credentials during login.
2. **FR2** The system shall display a form for users to submit maintenance queries or incident reports.
3. **FR3** The system shall assign a tracking number to submitted queries and notify users of successful submission.
4. **FR4** The system shall validate any attached images or files and include them with the query submission.
5. **FR5** The system shall notify users if a query submission fails due to network or system errors and allow a retry.

6. **FR6** The system shall validate user credentials during login.
7. **FR7** The system shall display the current status of all submitted queries.
8. **FR8** The system shall provide detailed progress updates (e.g., "in progress," "awaiting materials," "completed") for each query.
9. **FR9** The system shall provide estimated completion times and reasons for delays if queries take longer than expected.
10. **FR10** The system shall allow users to provide feedback on resolved queries or reopen them if the issue persists.

2.2.7 User Authentication and Role Management

Following are the requirements for User Authentication and Role Management:

1. **FR1** The system shall allow new users to register by entering the required data, which includes their name, email address, and password.
2. **FR2** Users will be able to access the system by entering their password and registered email address.
3. **FR3** Users should be able to reset their password by using the "Forgotten Password" option.
4. **FR4** The system should validate fields so that the password is going to be strong based on set policies such as length, containing uppercase letters, numbers and special characters.
5. **FR5** The system will grant access to specific functionalities based on the role assigned to the user.
6. **FR6** The admin will have the ability to add, modify, and remove user roles through the system.
7. **FR7** Sensitive modules and admin functions cannot be accessed by non-admin users.
8. **FR8** The system should permit the administrator to edit users accounts. This is to allow password resets and account cancellation.
9. **FR9** The system shall maintain log activity, especially at the admin level, to monitor the process.
10. **FR10** The system shall provide a feature for role-based permissions, where it would support polling generation, list properties and access to the admin dashboard.

2.2.8 Admin Dashboard

Following are the requirements for Admin Dashboard:

1. **FR1** Admin view of the system provide an overview dashboard that shows the activity and participation of the residents, for example, numbers for active residents, events, property listings, and under way maintenance requests.
2. **FR2** The system shall supply any emerging messages, notifications, or issues reported in real time as they begin and display on the dashboard.
3. **FR3** The system will allow the admin to sort and filter what is coming to the dashboard based on activity type, date or specific users.
4. **FR4** The system has to provide rapid access to important modules including voting polls, event management, and property operations.
5. **FR5** Critical activities such as outstanding maintenance issues or upcoming community voting dates will be displayed on the site.
6. **FR6** The system will have resources-sharing and property listings, which are pending approval.
7. **FR7** The report from the dash board of the system needs to be allowed to the admin for exportation in PDF or Excel or any file type.
8. **FR8** The application should enable admin to grant selective rights and to switch user roles from, for example, resident-to-admin or guest-resident.
9. **FR9** The admin will be able to access the system to search up citizens by using parameters like name, email, role, or account status.
10. **FR10** The system should enable the community manager to add or remove members from the group.
11. **FR11** The system must enable the admin to provide specific permissions and switch user roles (such as from resident to admin or guest to resident).
12. **FR12** The system should allow the administrator to either suspend or deactivate user accounts either permanently or temporarily.
13. **FR13** The system should allow the administrator to reset user passwords and alerting them of the change end.
14. **FR14** The system should maintain an audit log where all administrator will have the record of actions undertaken against users in terms of changes of user roles, account deactivation, and also resetting the password.

2.3 Non-Functional Requirements

2.3.1 Reliability

Failure Rate: The System shall has mean time failure between of minimum 1000 hours.

Failure Definition: The system will be unavailable due to error in any functionality and property listing, voting, resource sharing, communication does not operate.

Consequence: Residents will unable to use any operation and result in data loss.

Protection from Failure: The system shall have a back mechanism to prevent it from any single failures.

Error Detection Strategy The system shall monitor all the logs continuously in case of error detection.

Error Correction Strategy The system shall have automated backup mechanism to recover single operation with in 5 minutes.

2.3.2 Usability

Ease of Learning: The system shall designed in a way by which user can learn to use within 10 minutes.

Ease of Use: The system shall has user-friendly interface and it will allow user to done a task with minimal effort.

Examples

USE-1 The system shall allow the user to retrieve the property listing with a single interaction.

USE-2 The system shall guide a new user step-by-step to buy new property process.

USE-3 The system shall display an error message when a user enter incorrect or invalid details.

2.3.3 Performance

Response Time: The system shall respond to the user requests with in 3 seconds.

Scalability: The system shall handle the user when increases without any degradation in performance.

Load Handling: The system should be able to handle the loads when on its peak number of users.

Resource Utilization: The system shall use the resources RAM, CPU and GPU if exist in user system.

Throughput: The system shall be able to do maximum number of transaction.

Examples

PER-1: The system shall load 90 percent of all the web pages within the 4 seconds from the time the user requests the page over 20 Mbps or more fast internet.

PER-2 The system shall be able to handle at least 500 residents without performance degradation.

PER-3 The system shall be able to handle at least 50 transaction during system peak time.

2.3.4 Security

Data Encryption: The system shall encrypt all sensitive data and secure them in a block chain.

Authentication and Authorization: The system shall use strong authorization and authentication methods i.e two factor, MFA etc.

Access Control: The system shall allow users to perform activities according to their role.

Audit Trails: The system shall audit all transactions and trail related activities.

Examples

SEC-1: The system shall use standard algorithms to protect the sensitive information.

SEC-2: The system shall have a multi factor authentication or 2FA authentication for the admin login.

SEC-3: The system shall maintain the logs of users and have at least 6 months audit details.

Chapter 3

System Overview

3.1 Architectural Design

The CommuneX system is designed to manage several aspects of Neighborhood and Community automation which include the buying and selling of the properties, the decentralized voting system for the community decisions, property documents stored on a block chain to maintain the security and transparency and resource sharing among the community members. By decomposing these modules into the system we will be providing the complete web application for the community purpose. These modules integrates with each other maintain the desired functionality.

3.1.1 System Architecture Design

The architecture diagram for CommuneX outlines a system that integrates various aspects of community management. Users interact through a client interface, such as mobile or web applications, which connect to a backed powered by Node.js. The server processes requests, manages data flow, and interacts with both a blockchain for secure decentralized operations (like voting or property transactions) and a database that stores community-related information. The design ensures secure data handling and efficient management of community services like communication, voting, and shared resources.

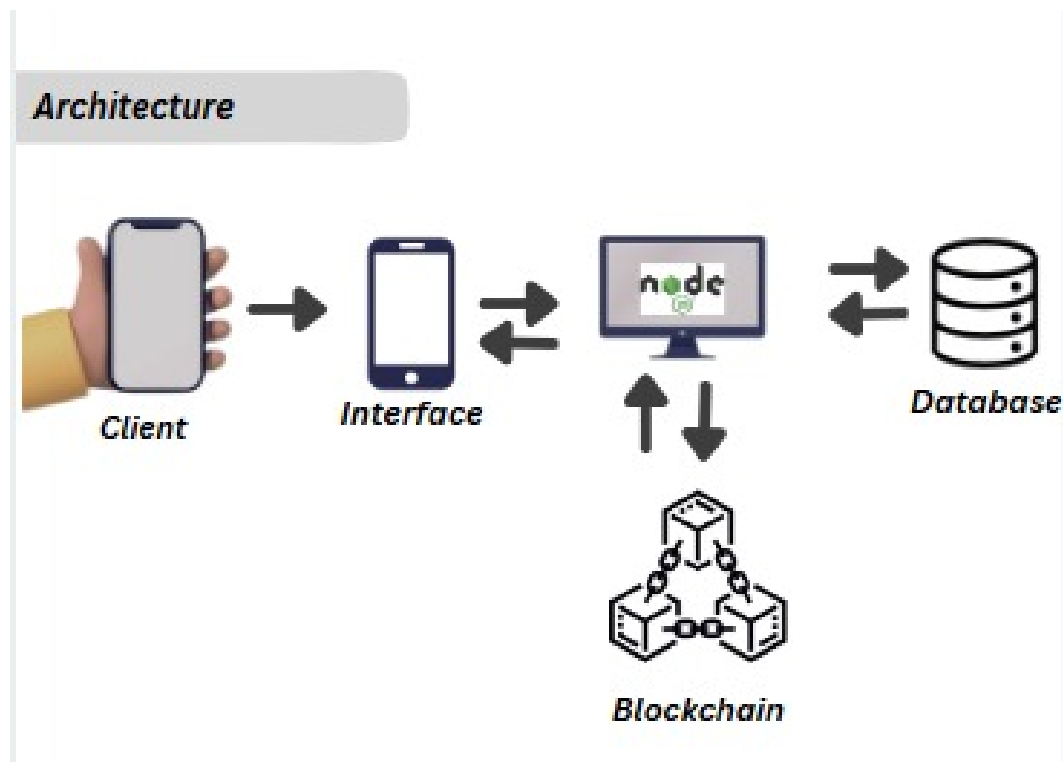


Figure 3.1: System Architecture diagram

3.2 Design Models

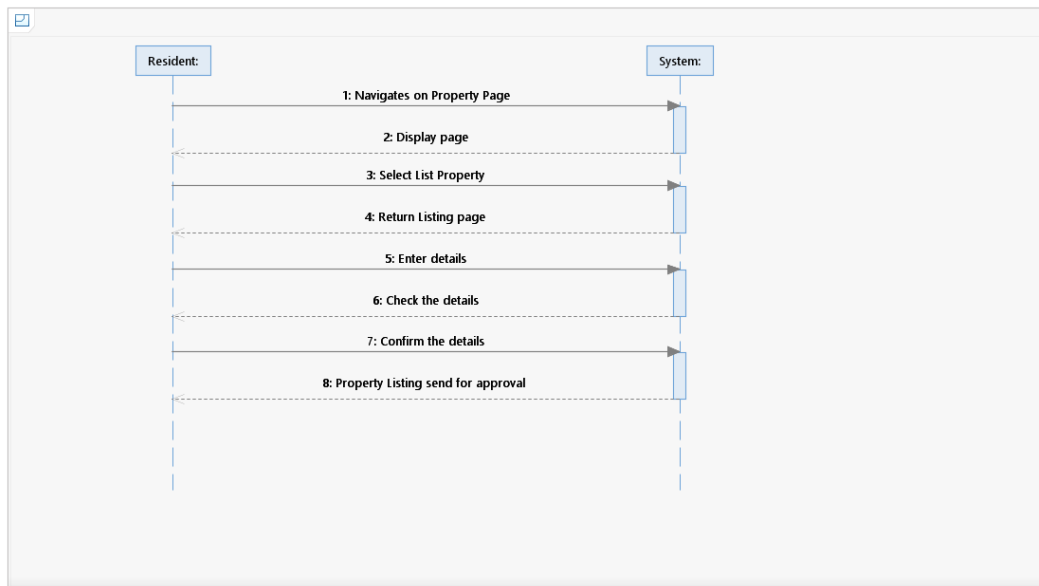


Figure 3.2: Property Listing

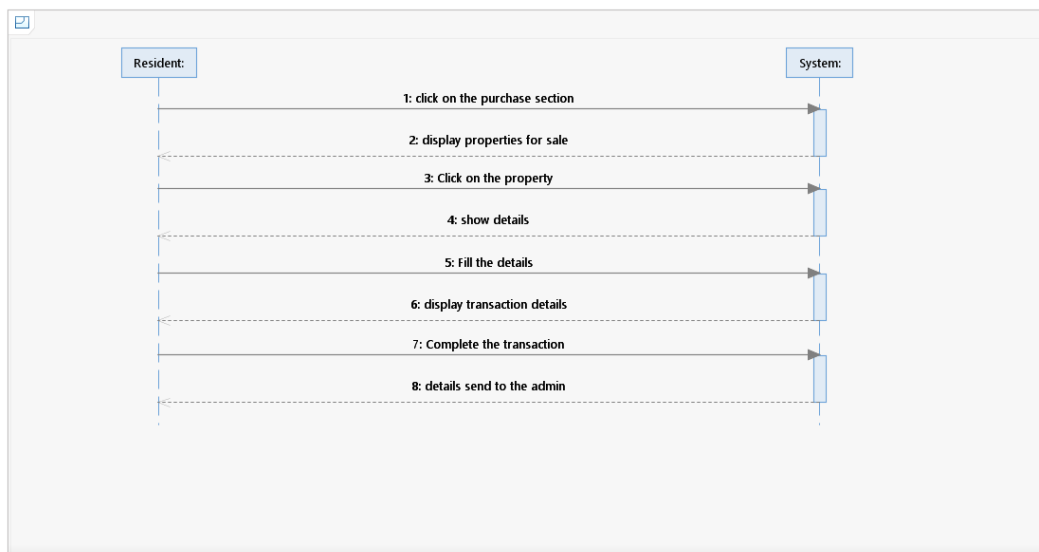


Figure 3.3: Purchase Property

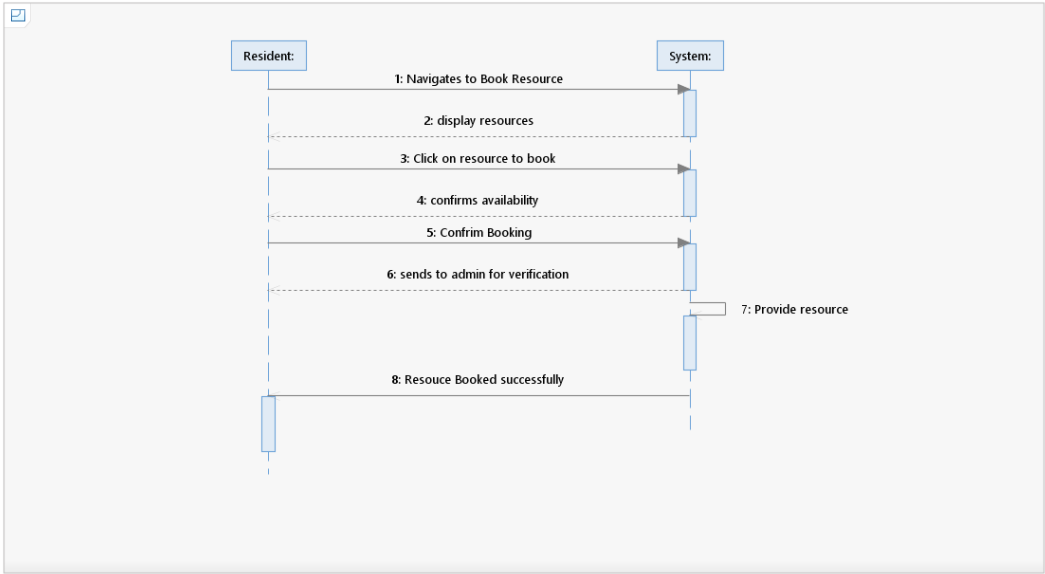


Figure 3.4: Resource Booking

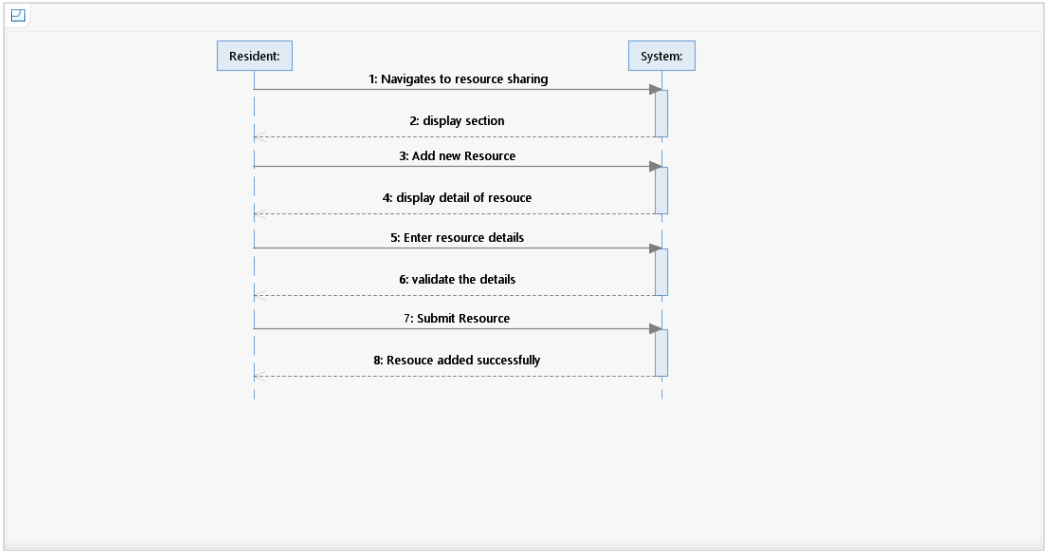


Figure 3.5: Resource Listing

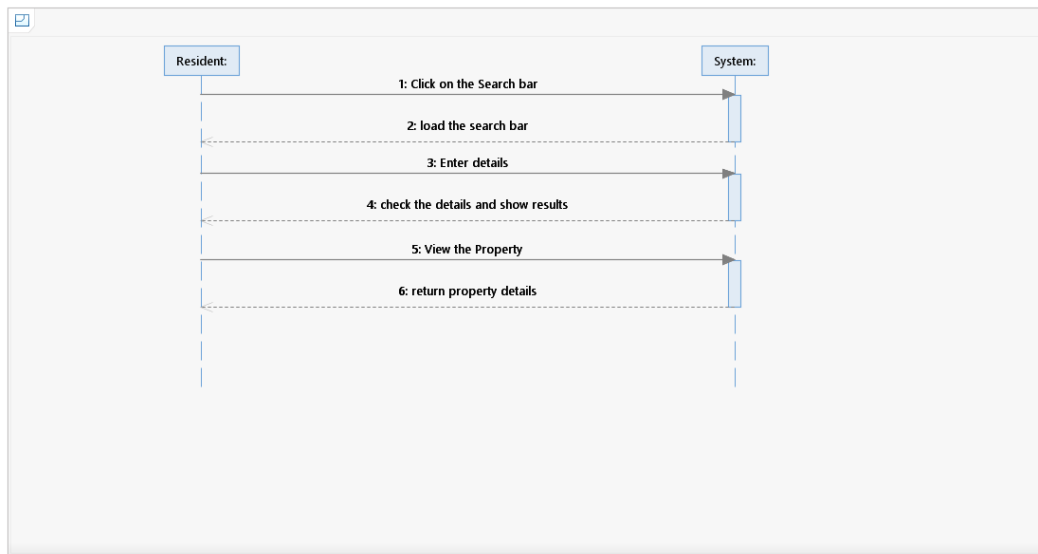


Figure 3.6: View and Search Property

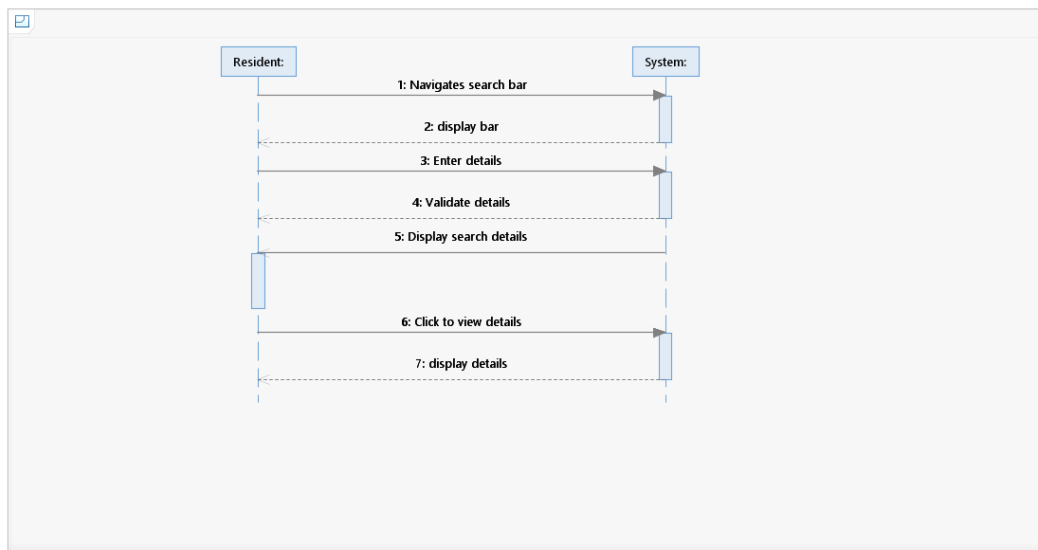


Figure 3.7: view and search resource

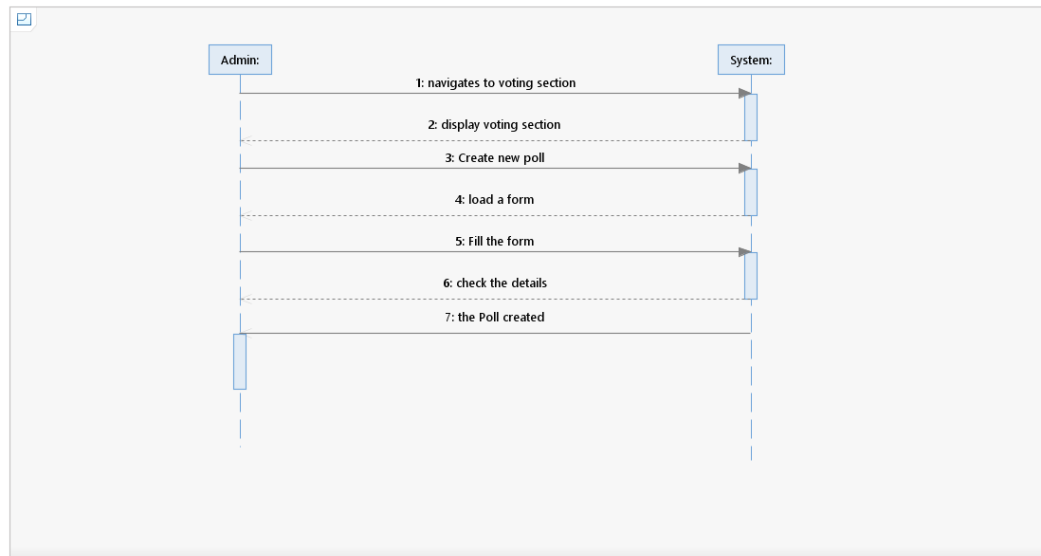


Figure 3.8: voting creation

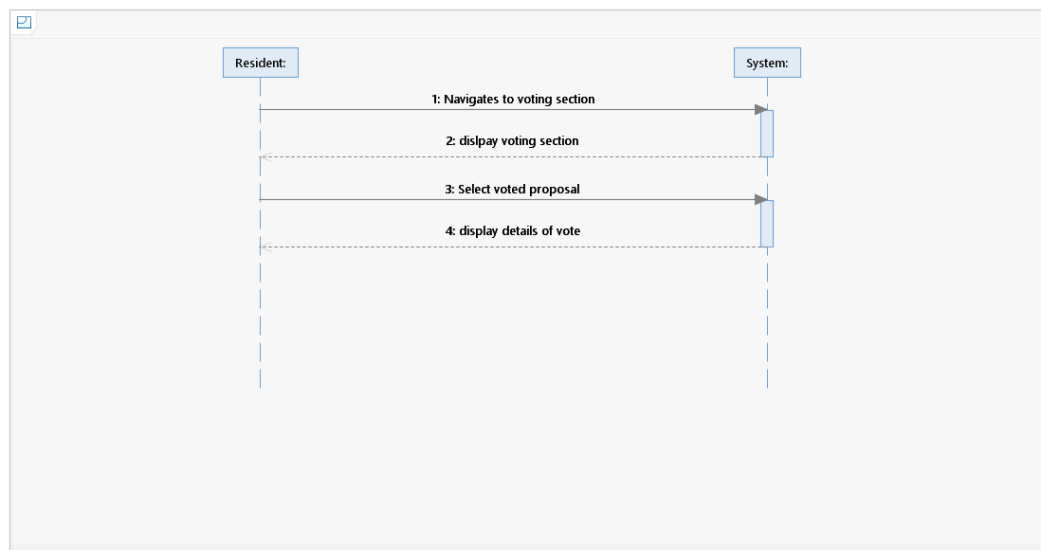


Figure 3.9: Voting Results



Figure 3.10: voting

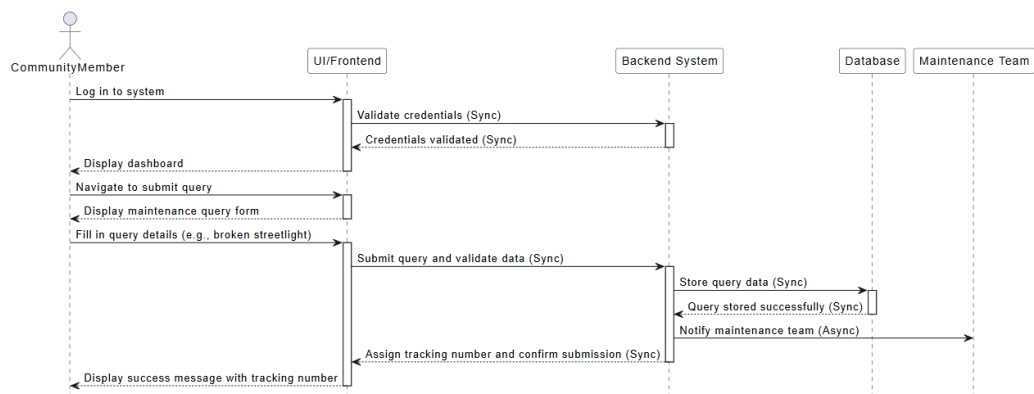


Figure 3.11: Event Management

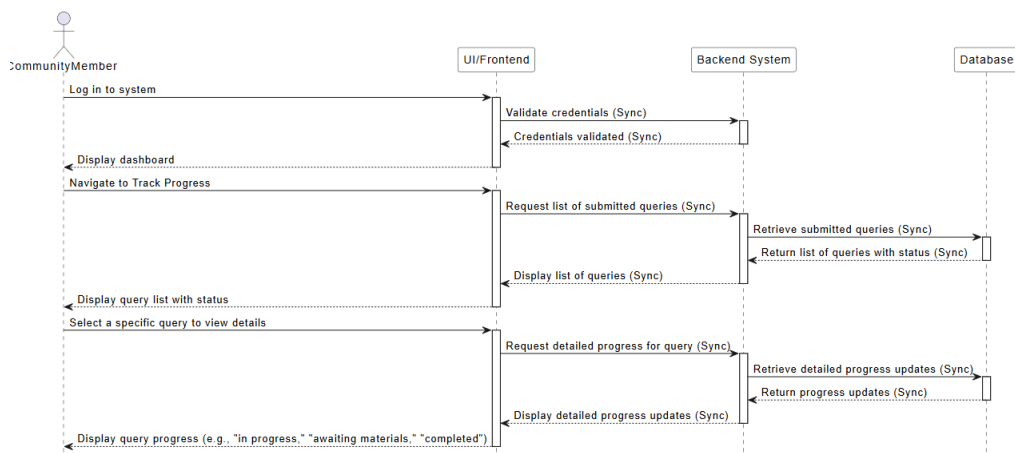


Figure 3.12: Event Recommendation

3. System Overview

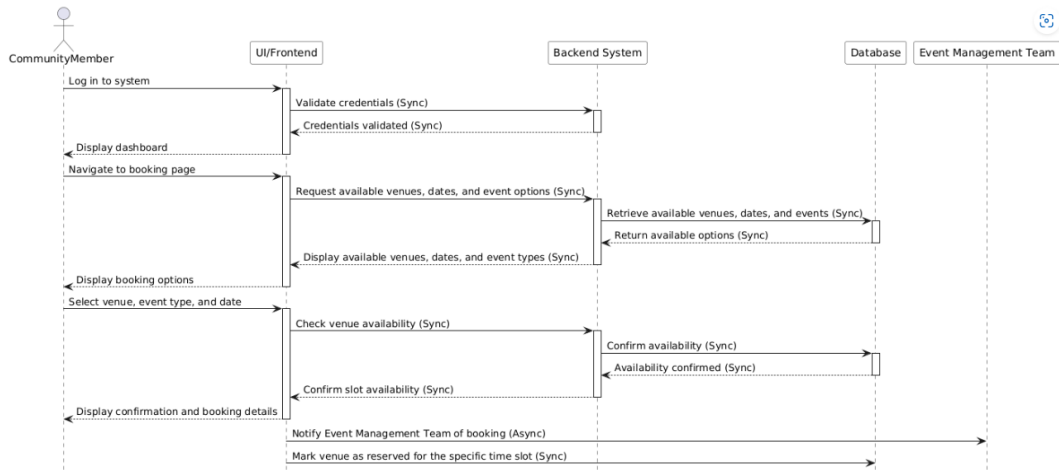


Figure 3.13: Community News and Announcements

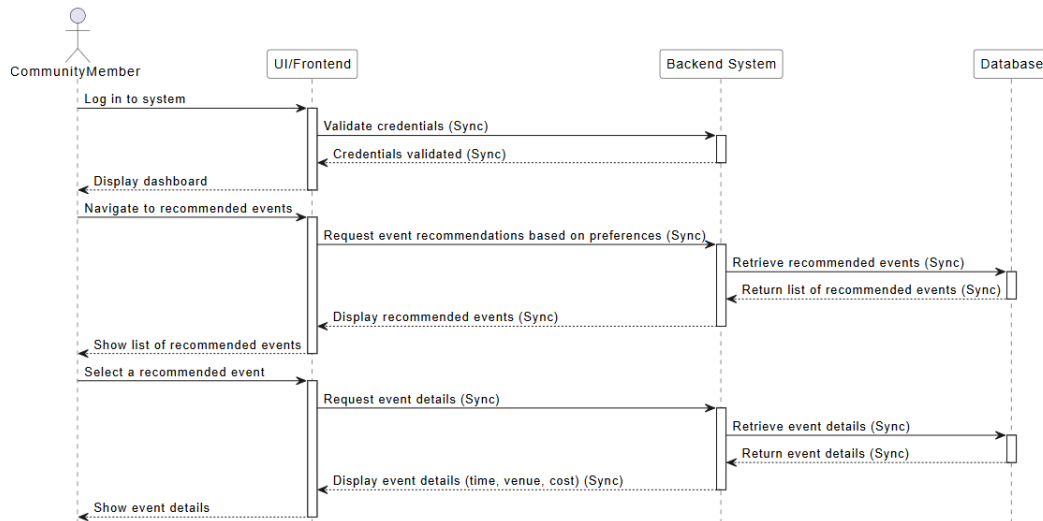


Figure 3.14: Private Messaging

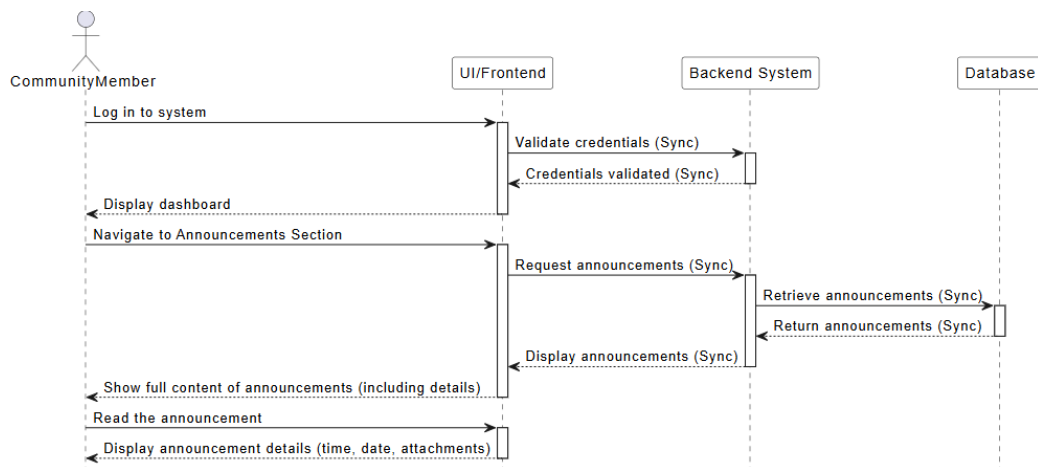


Figure 3.15: Submit Maintenance

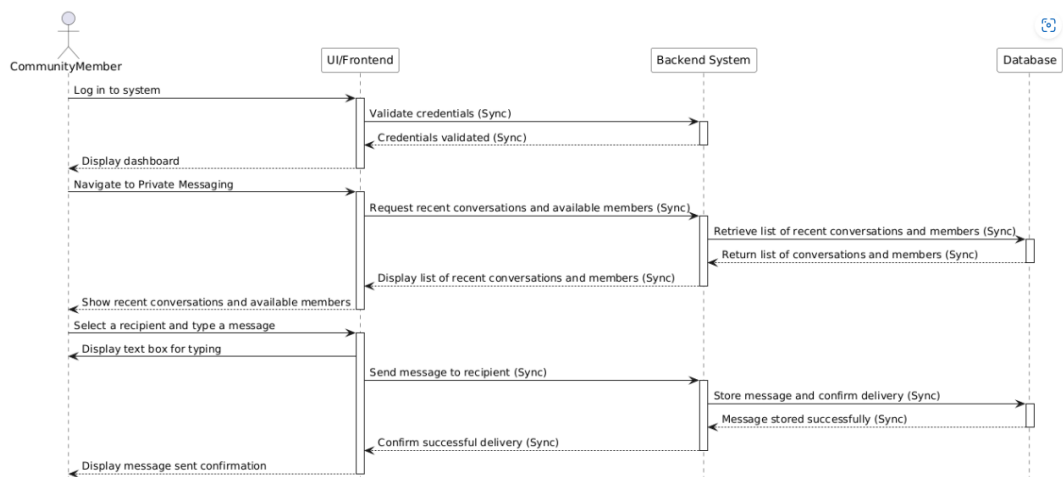


Figure 3.16: Track Progress

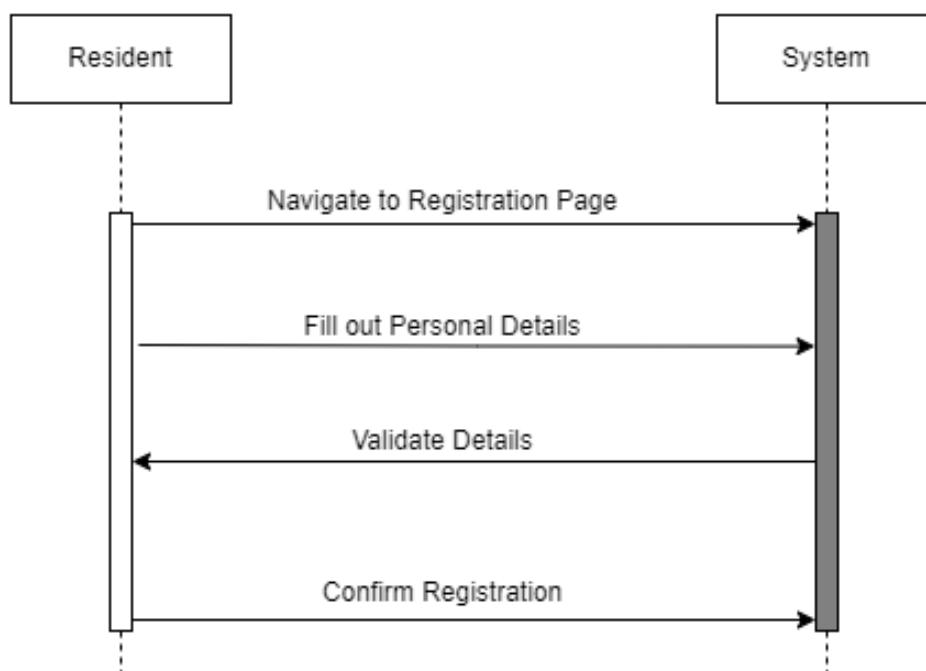


Figure 3.17: User Registration

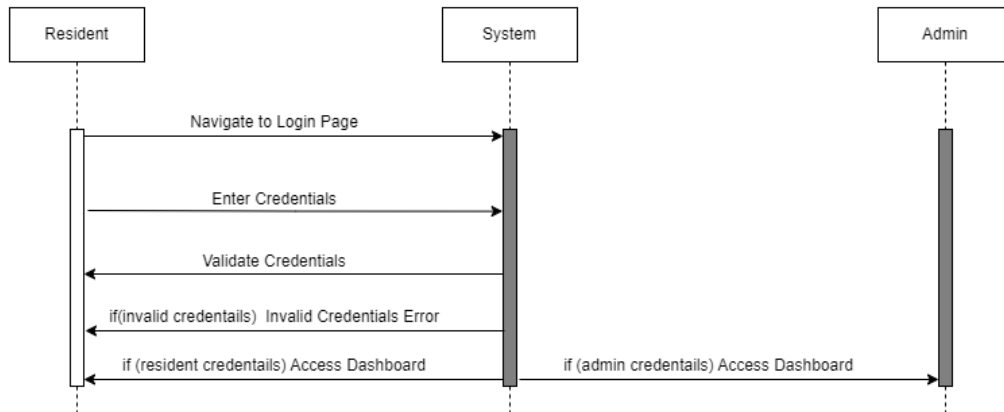


Figure 3.18: User Login and Role Assignment

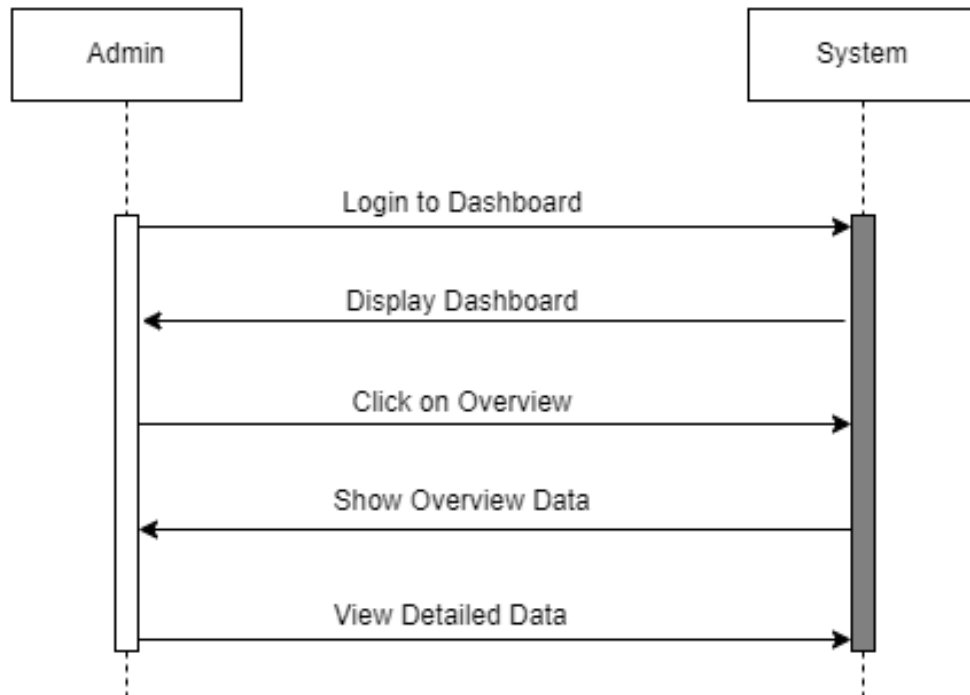


Figure 3.19: Admin Dashboard

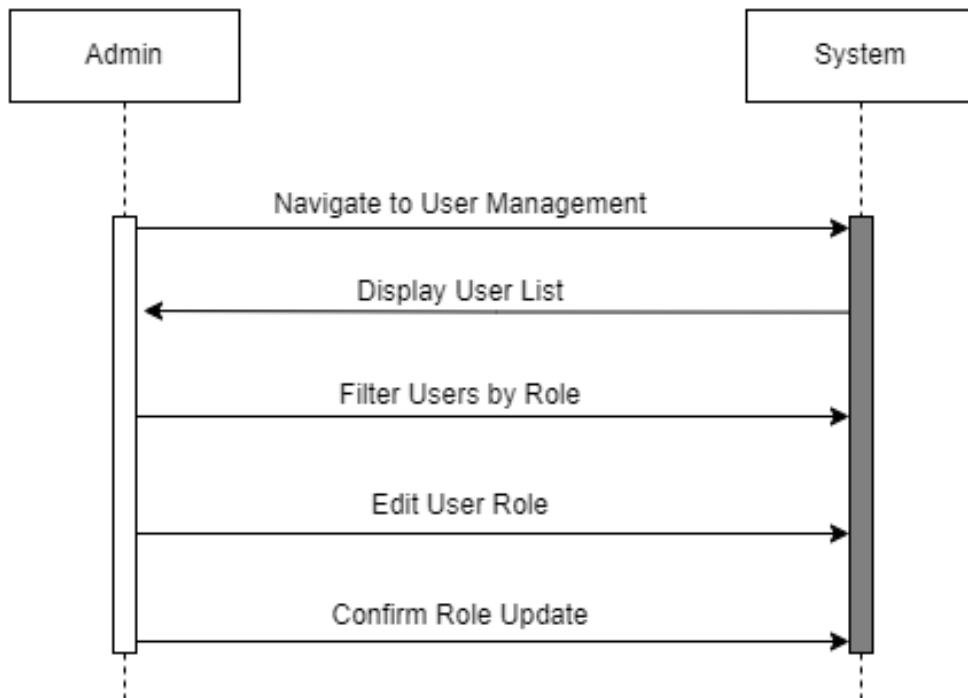


Figure 3.20: Manage Users and roles

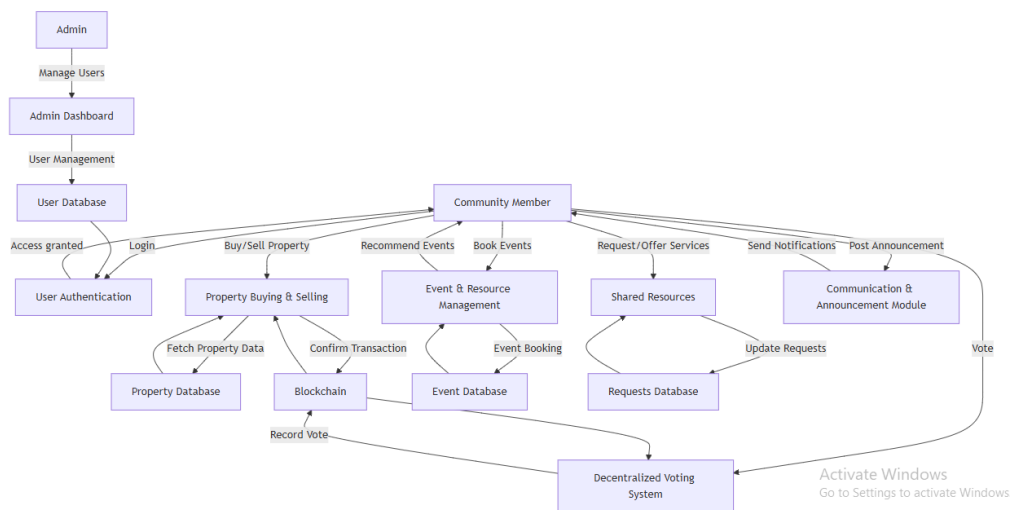


Figure 3.21: Data Flow Diagram for CommuneX

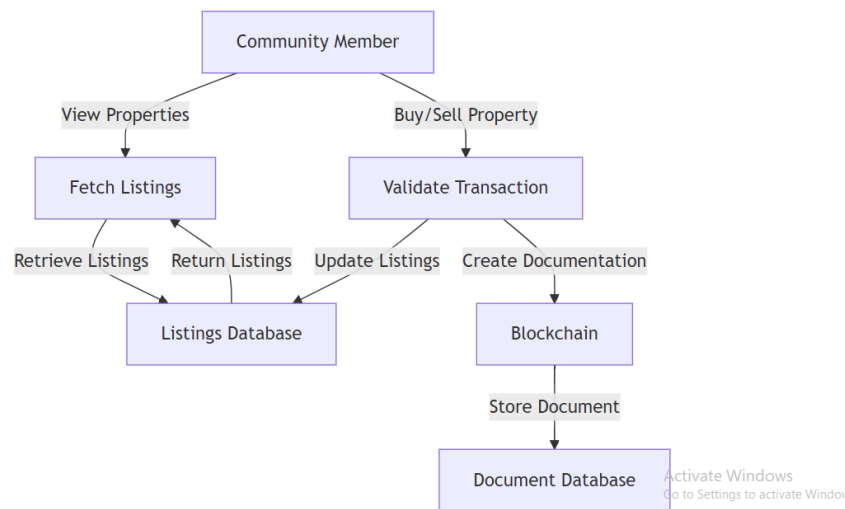


Figure 3.22: Data Flow Diagram for CommuneX

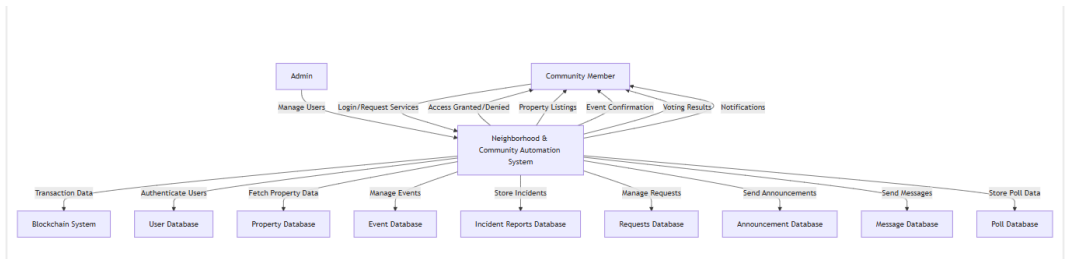


Figure 3.23: Data Flow Diagram for CommuneX

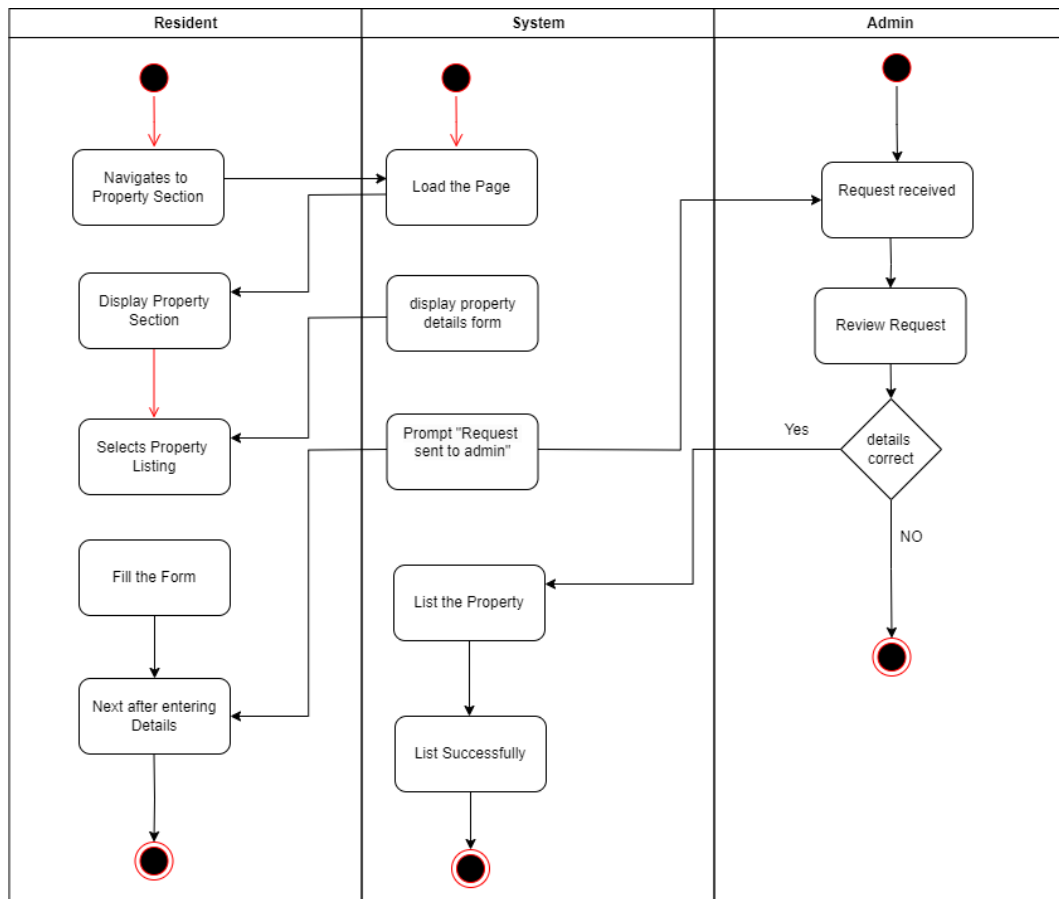


Figure 3.24: Property Listing Activity diagram

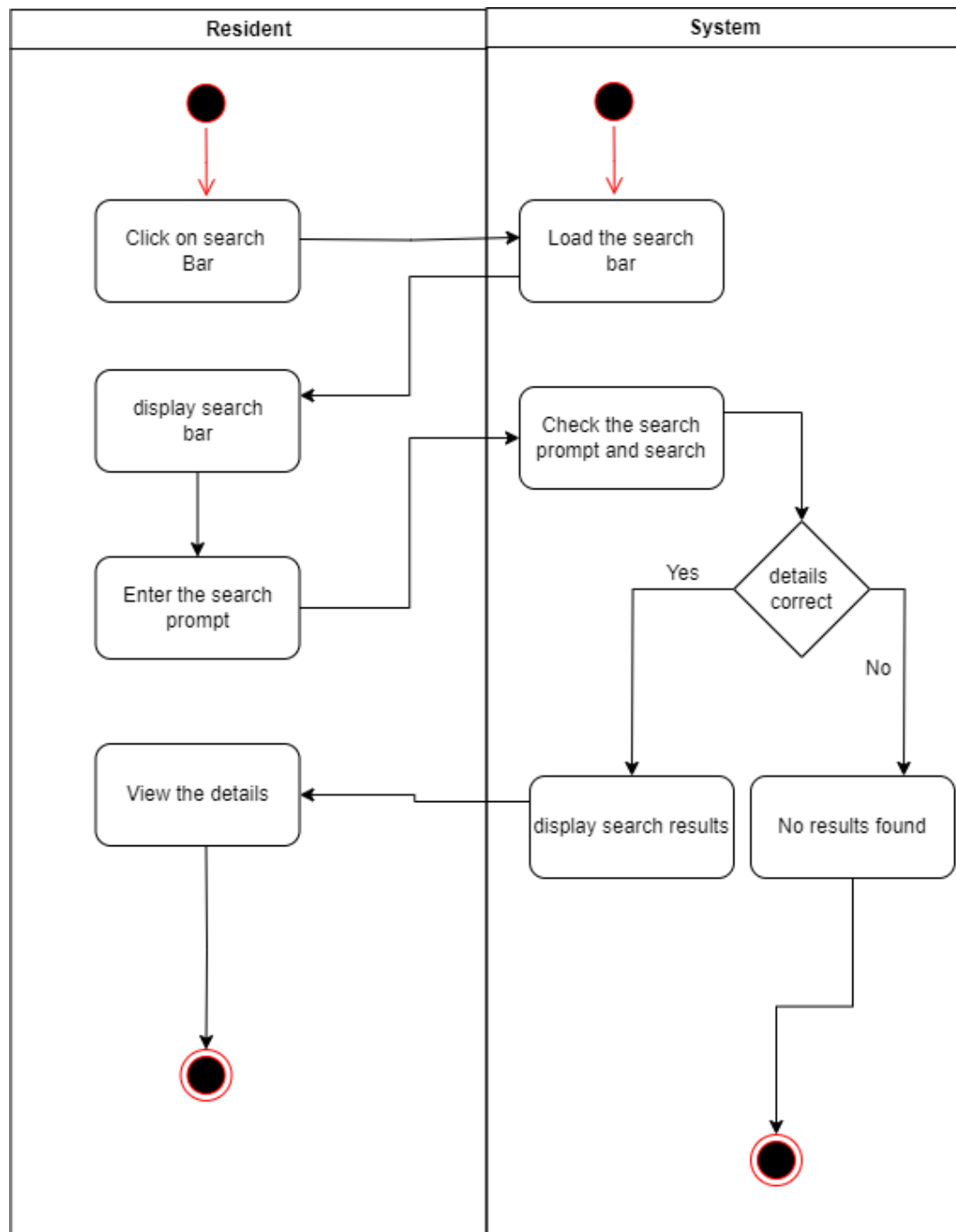


Figure 3.25: Property View and Search Activity diagram

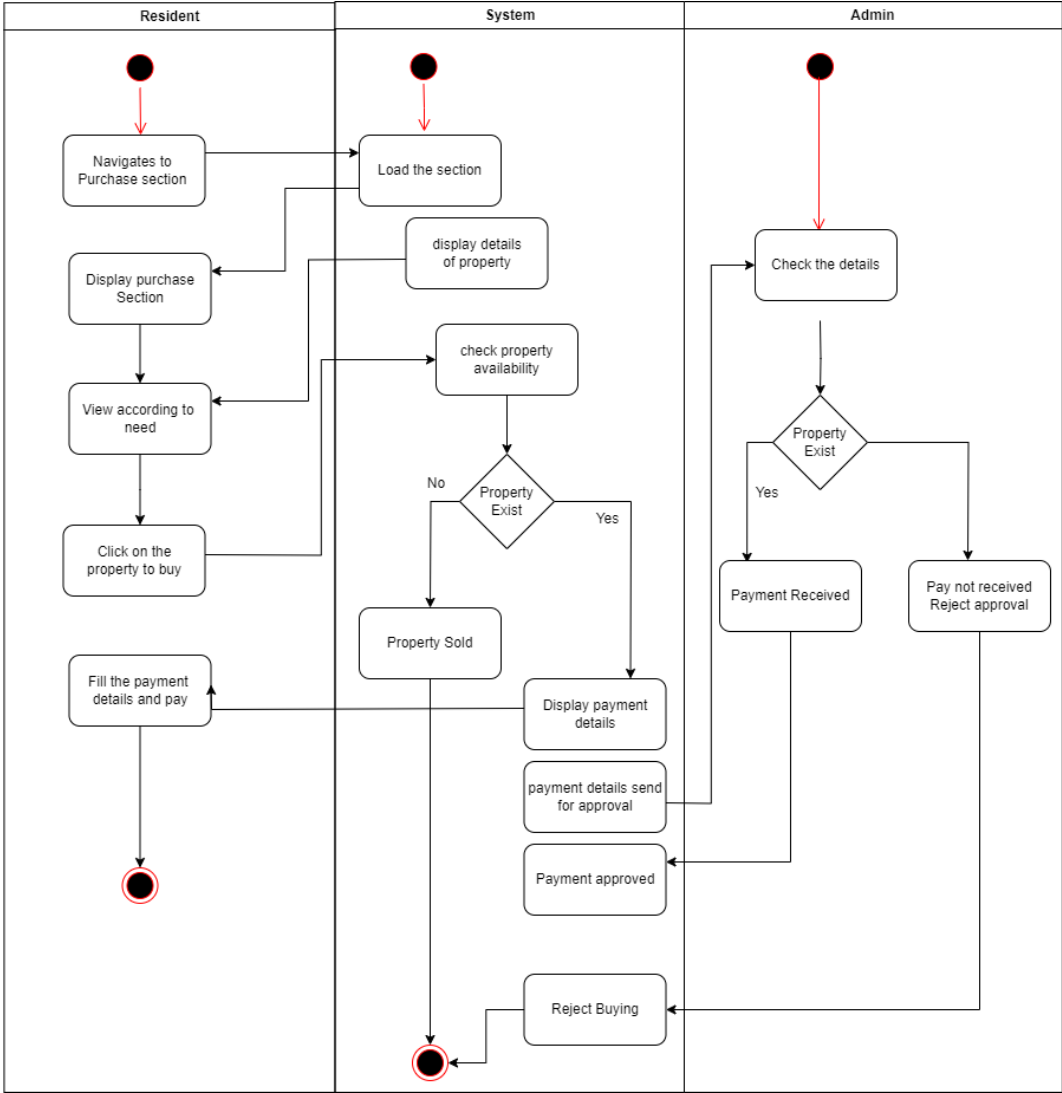


Figure 3.26: Purchase Property Activity diagram

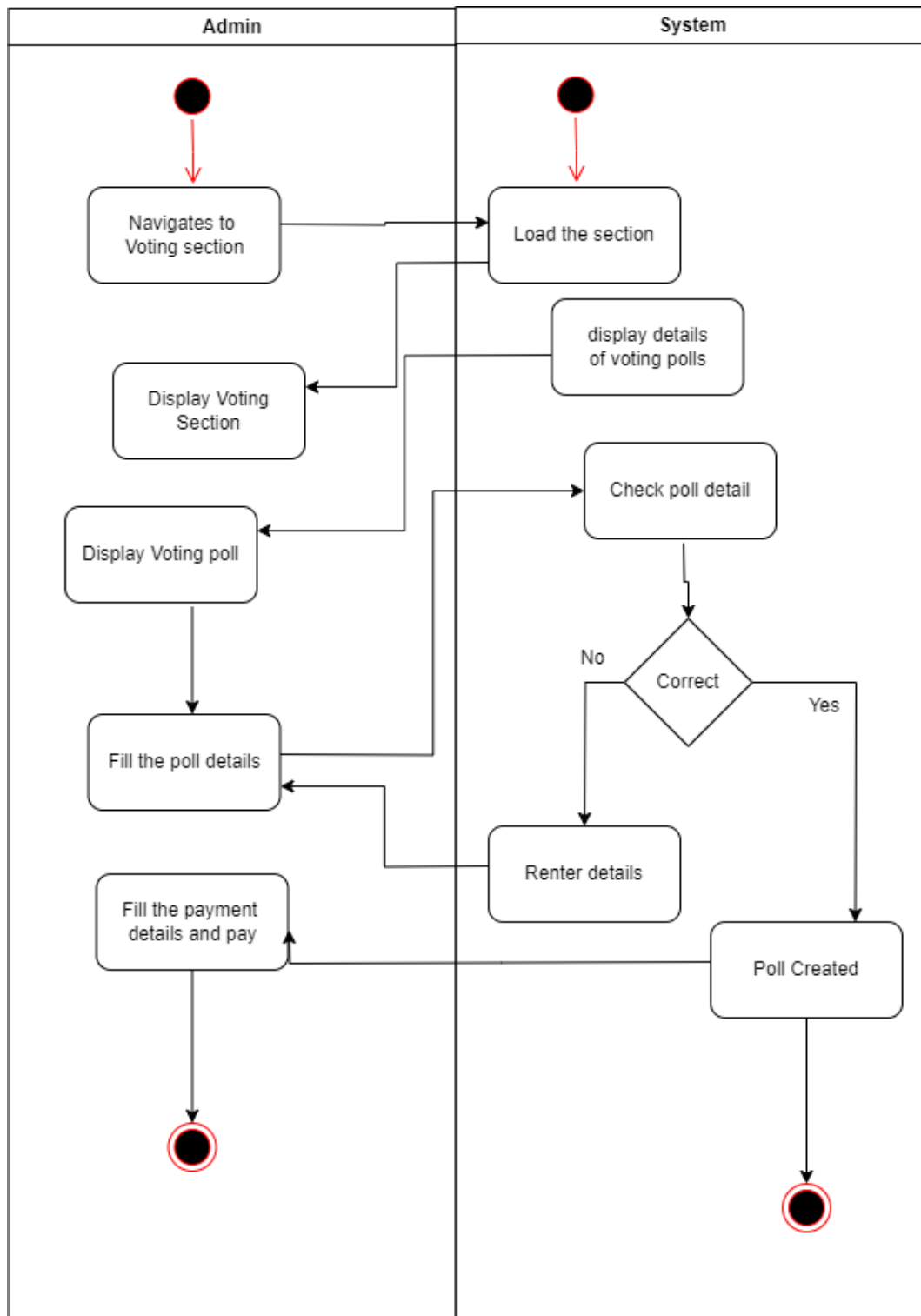


Figure 3.27: Voting Creation Activity diagram

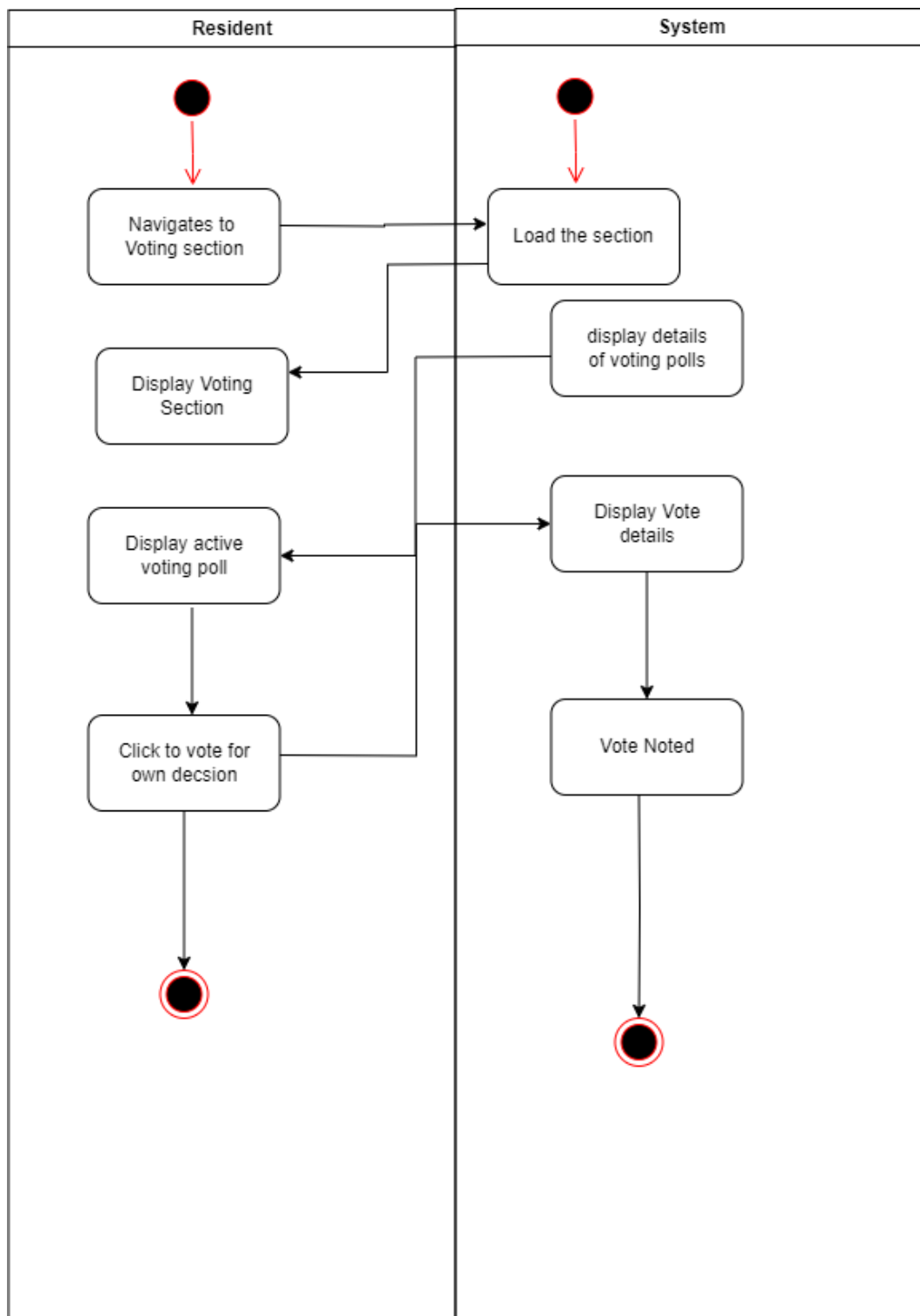


Figure 3.28: Voting Activity diagram

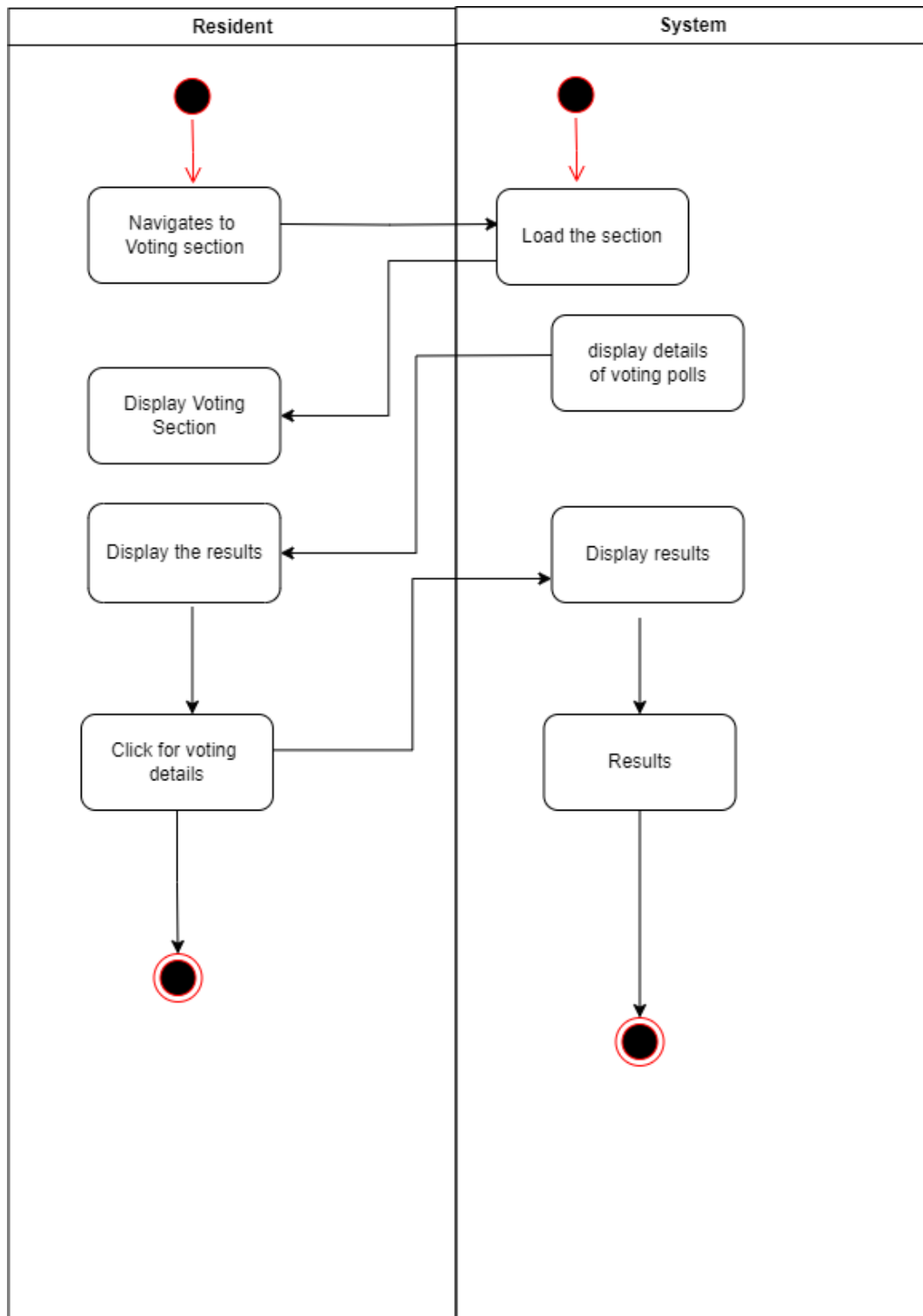


Figure 3.29: Voting Results Activity diagram

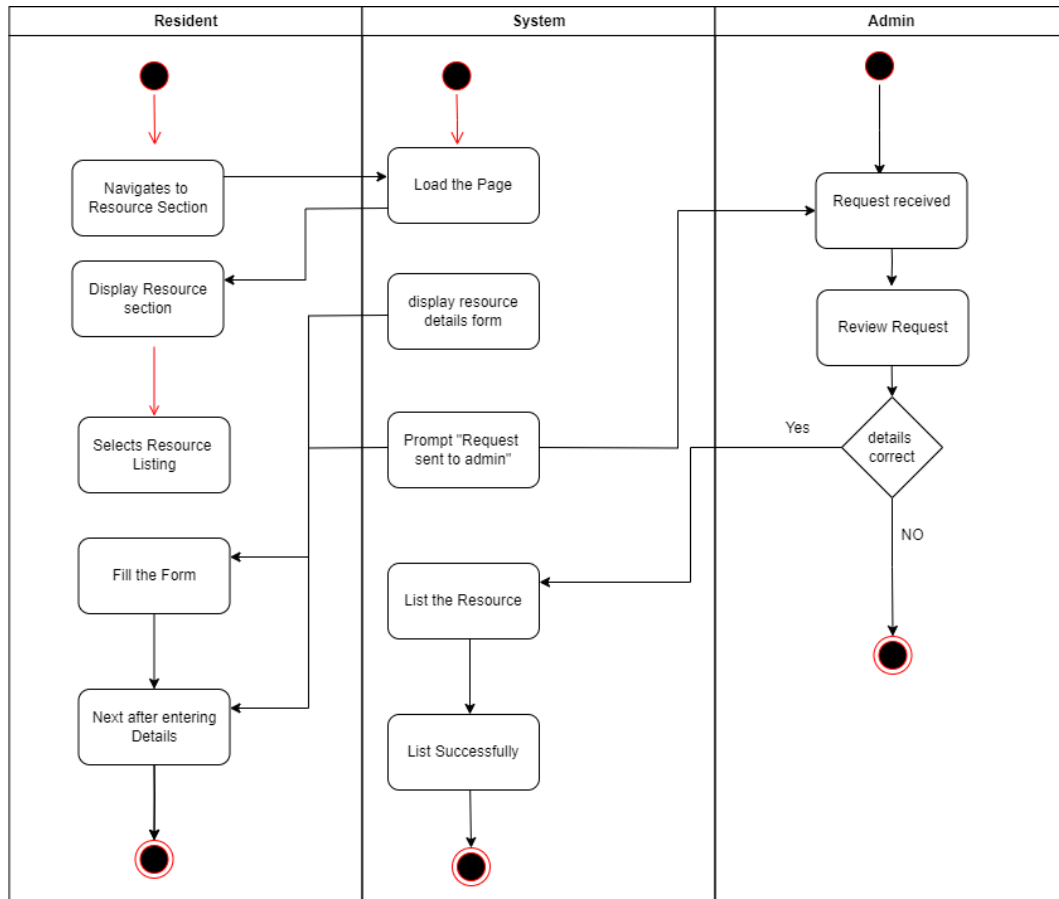


Figure 3.30: Resource List Activity diagram

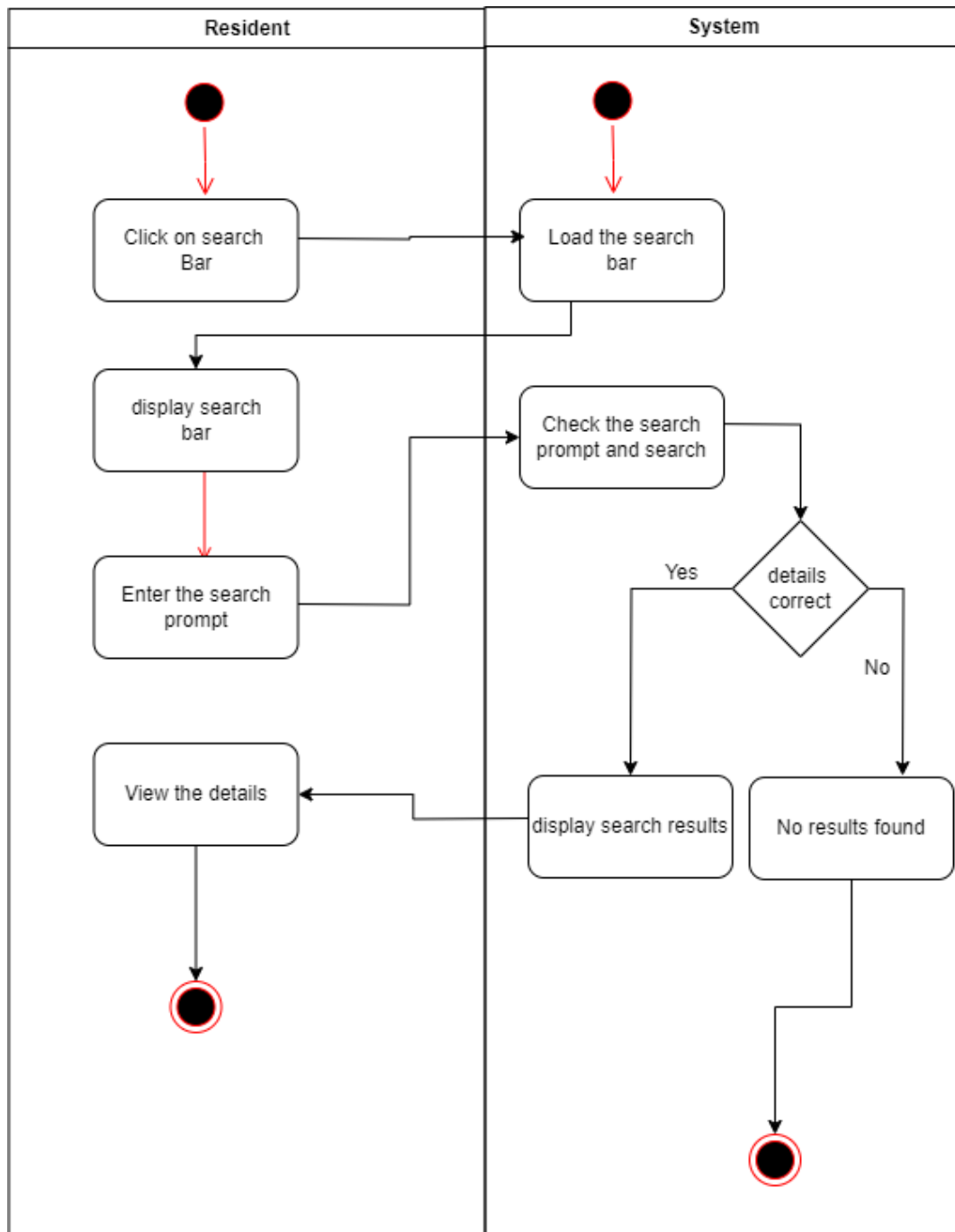


Figure 3.31: Resource Search and view Activity diagram

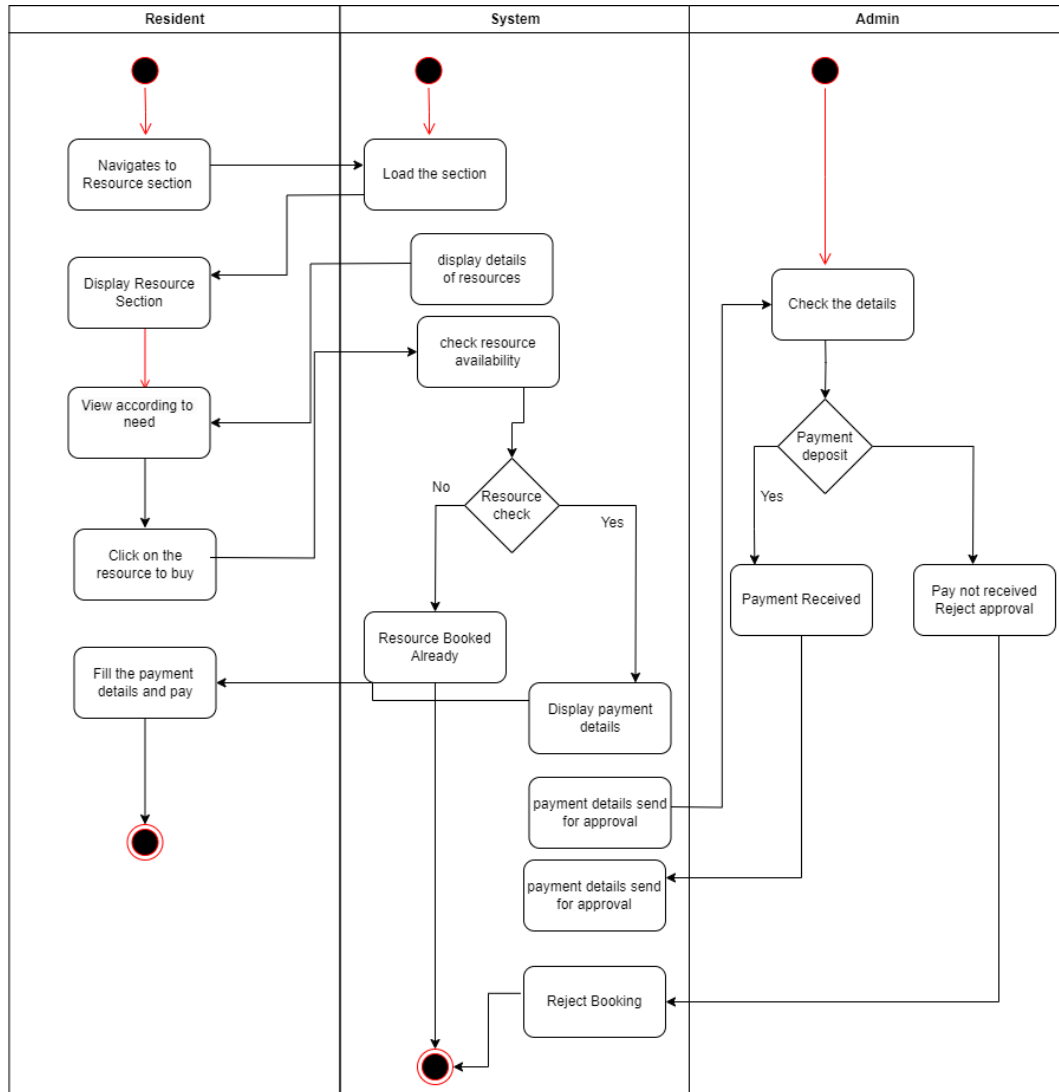


Figure 3.32: Resource Booking Activity diagram

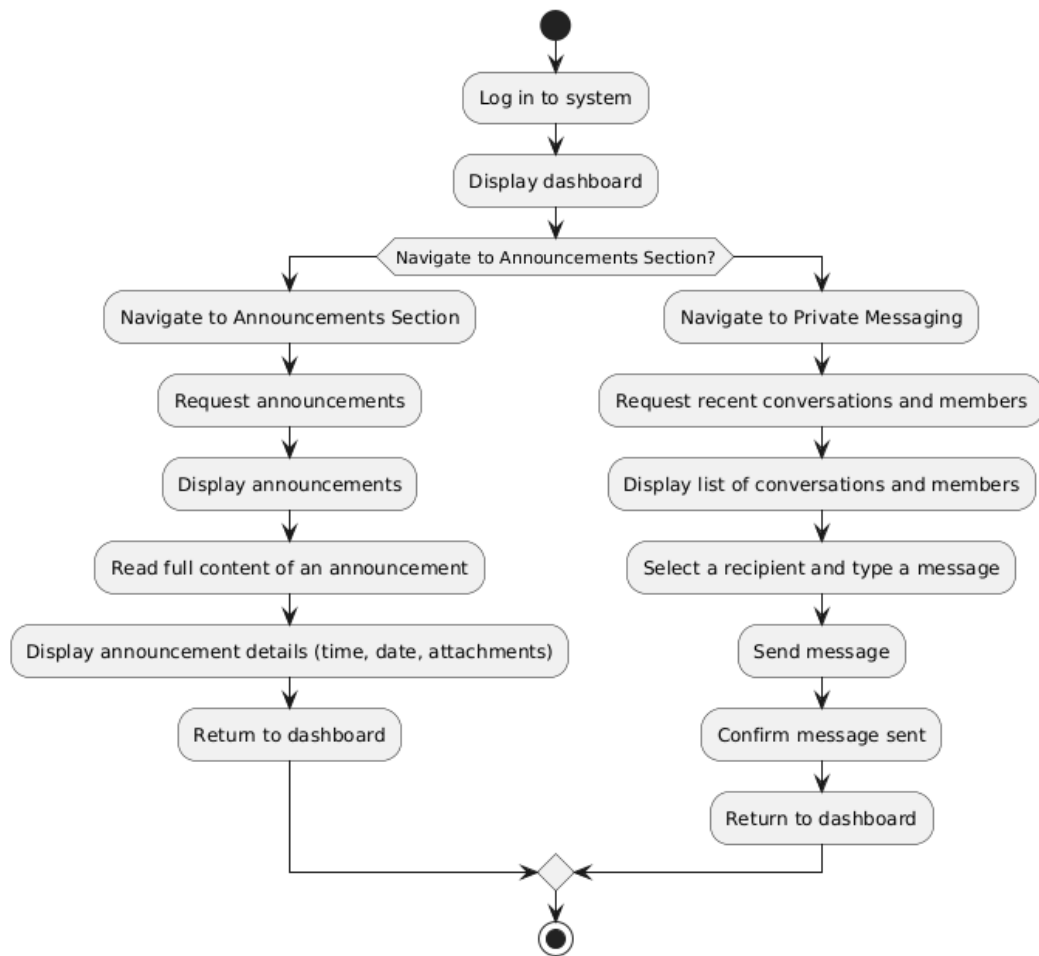


Figure 3.33: Communication and Announcements Activity diagram

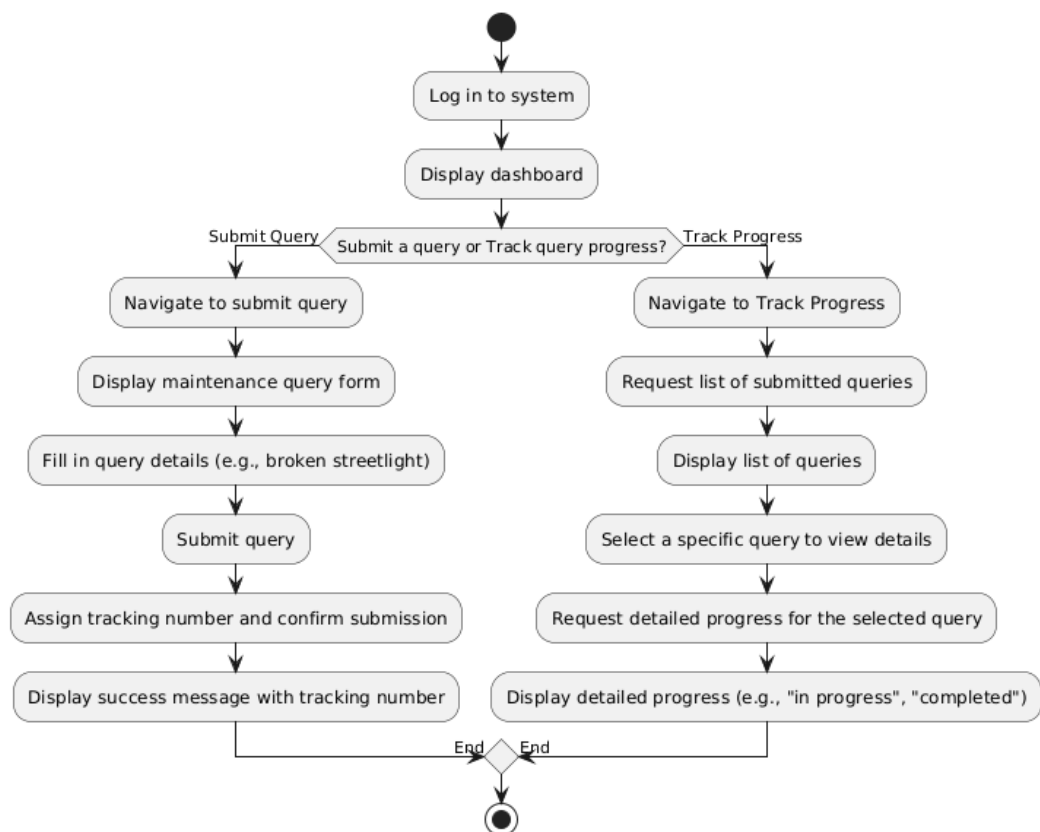


Figure 3.34: Maintenance Activity diagram



Figure 3.35: Resource List Activity diagram

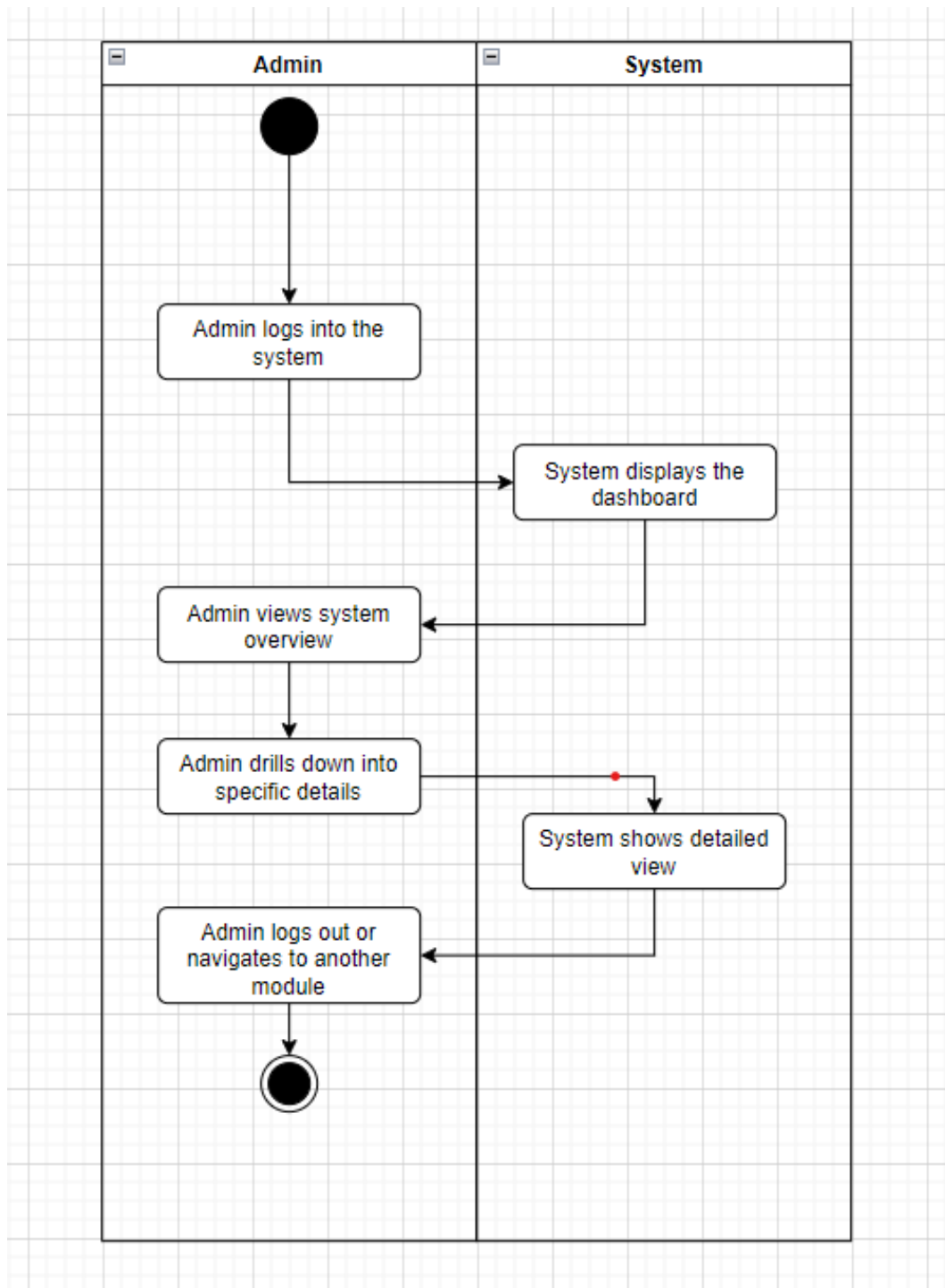


Figure 3.36: Admin activity diagram

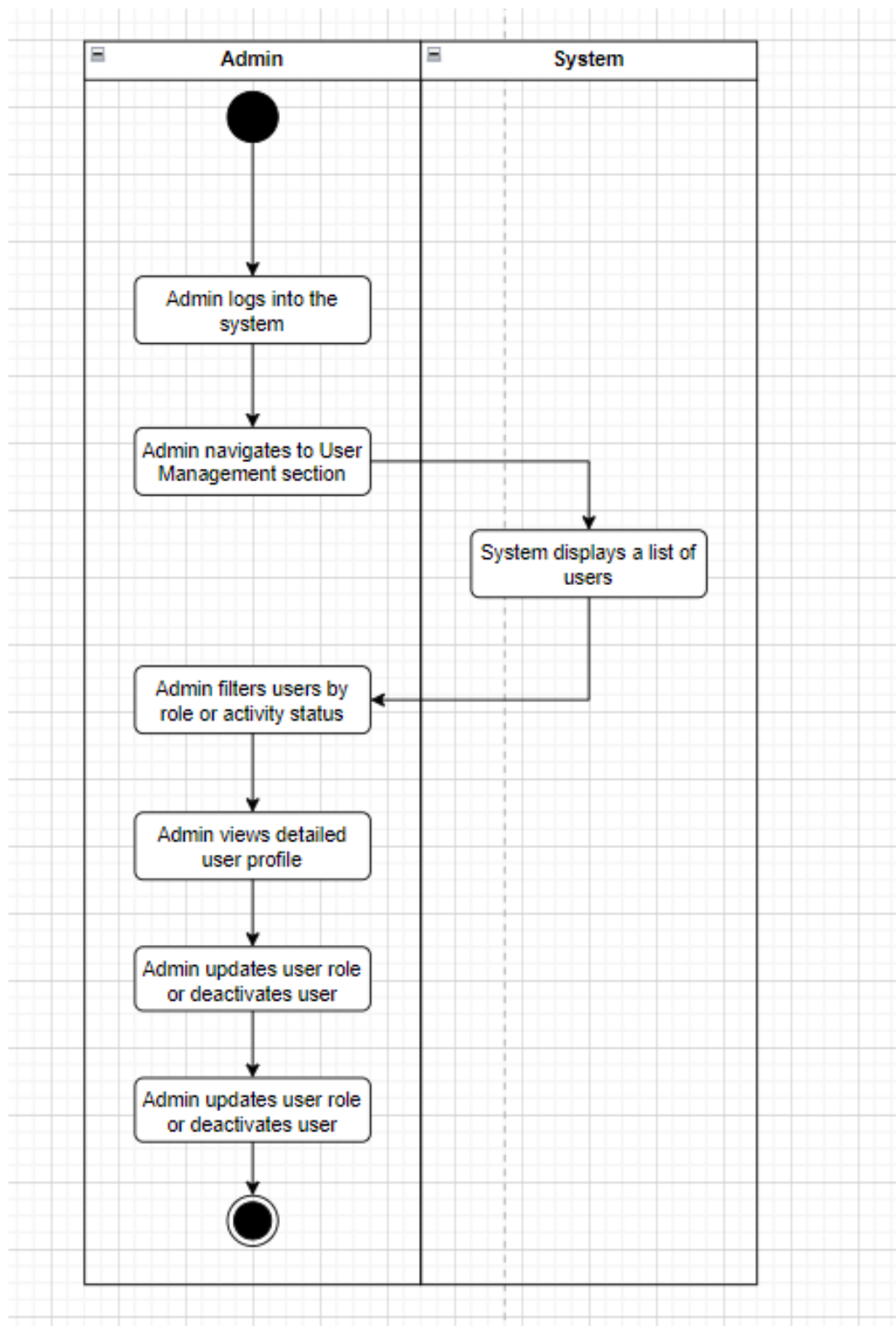


Figure 3.37: User Management activity diagram

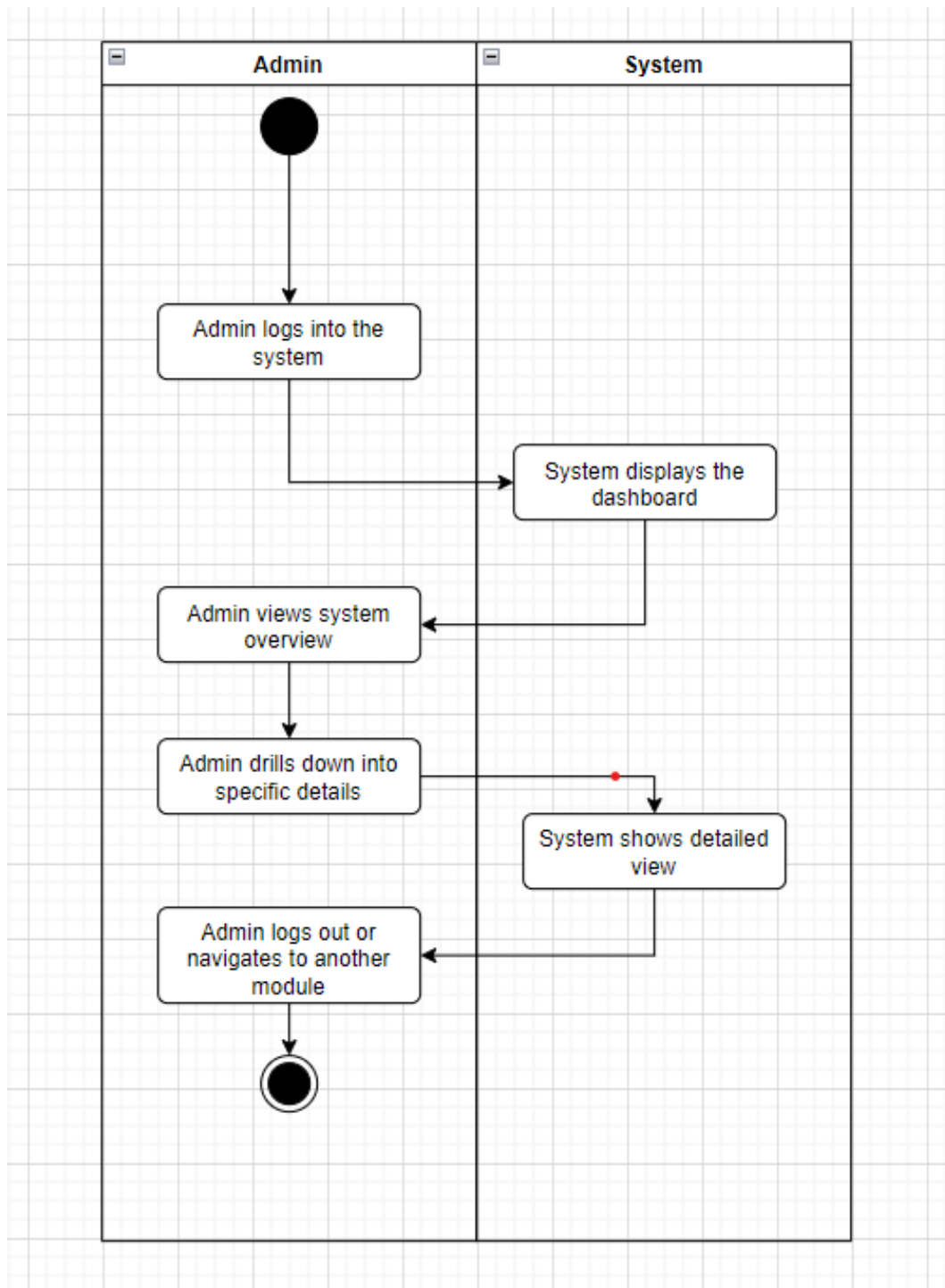


Figure 3.38: Register User activity diagram

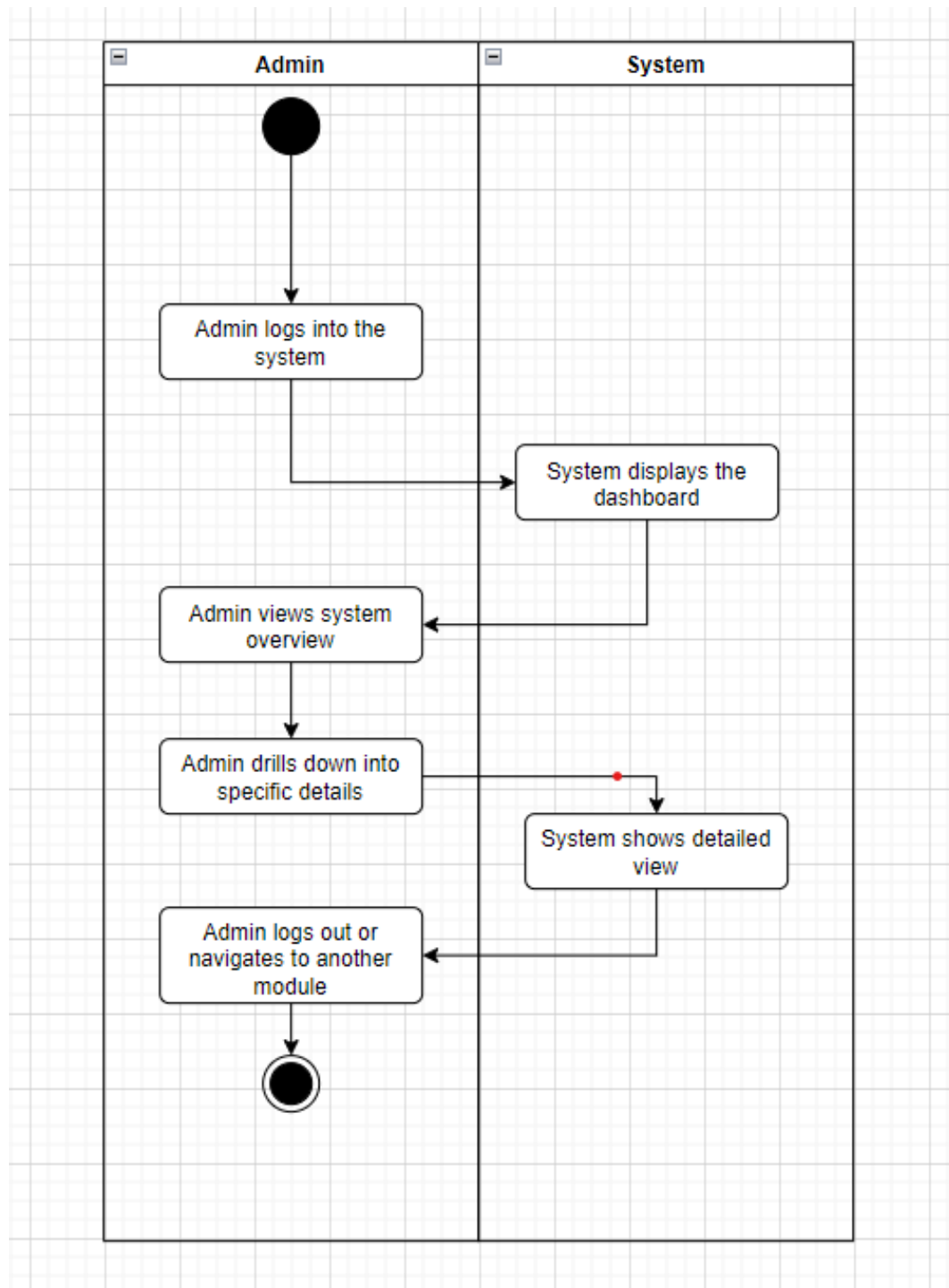


Figure 3.39: User role and assignment activity diagram

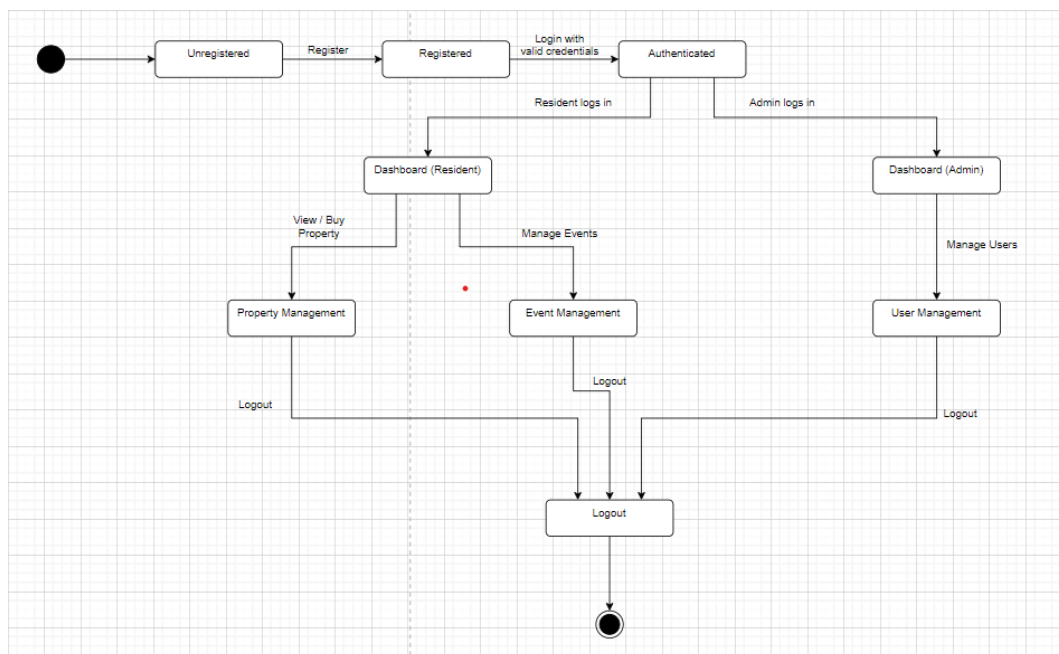


Figure 3.40: State Transition diagram

3.3 Data Design

The community automation system information domain is converted into the data structures which help us to store the data processing and the entities like properties, voting creation, voting results, voting, resources, community engagement, private messaging and resource bookings. These entities store in non structural SQL like MongoDB while the property transaction, property papers and the other content stored on block chain. By arranging the data in this way our system ensure data transparency, security and information protection to improve the user experience.

The database design for the community automation system will be:

3.3.1 User Management

It stores data of each users including userID, name, email, password, role and the contact information. The userID assign to the user automatically by the mongoDB.

3.3.2 Property Management

The property management contains the list of the property for sale and the transaction details. For the property listing the entities are ownerID, location, size, price details, photos, status for availability.

The transaction details include propertyID, buyerID, sellerID, date of transaction, price and the status.

3.3.3 Decentralized Voting system

The voting system contains two databases one for the voting creation and other is for casting a vote.

The voting proposal creation includes votingID, title, description of voting, options for vote, deadline and status of the voting.

The voting includes proposalID, voterID, voting option and the timestamp for the voting.

3.3.4 Resource Sharing

The resource sharing contains two databases. One is for resource listing and other is for resource booking.

The resource booking contains each resource ID, ownerID, type, description, availability, pho-

tos and status.

The resource booking includes ID, resourceID, userID, booking period and the status.

Bibliography